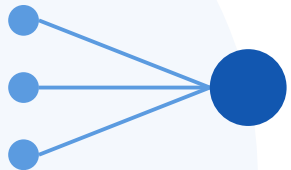


Why Do We Need Neural Networks?

From weights and bias to the perceptron, its limits, and the birth of deep networks



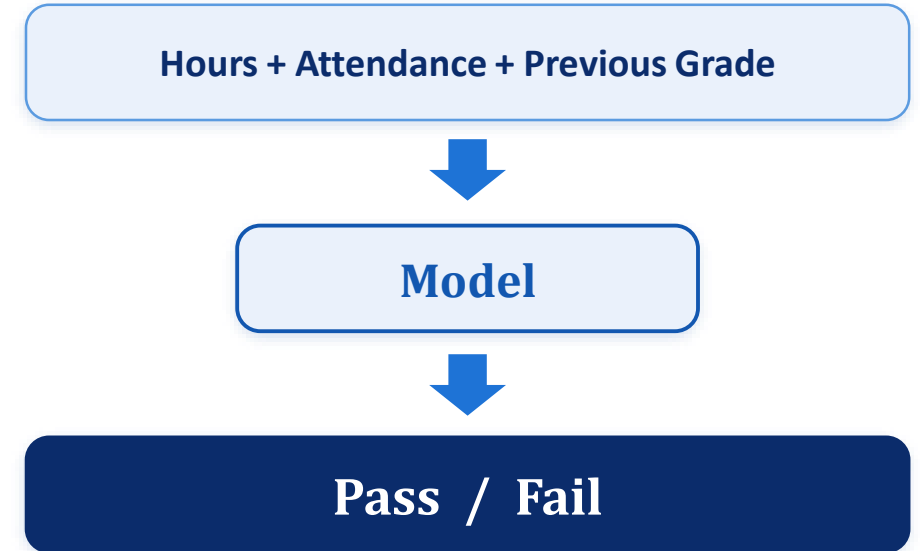
Nancy Ahmed

How Can a Machine Make a Decision From Several Inputs?

Goal: predict whether a student will Pass or Fail using three signals.

Student	Hours	Attendance	Prev. Grade	Result
A	2	60%	55	Fail
B	5	80%	70	Pass
C	8	95%	85	Pass
D	1	50%	40	Fail

The real question: how should the model combine these inputs?



Not All Inputs Are Equally Important

Student A

Hours studied 8

Attendance 95%

Previous grade 85

Very likely to PASS

Student A'

Hours studied 8

Attendance 50%

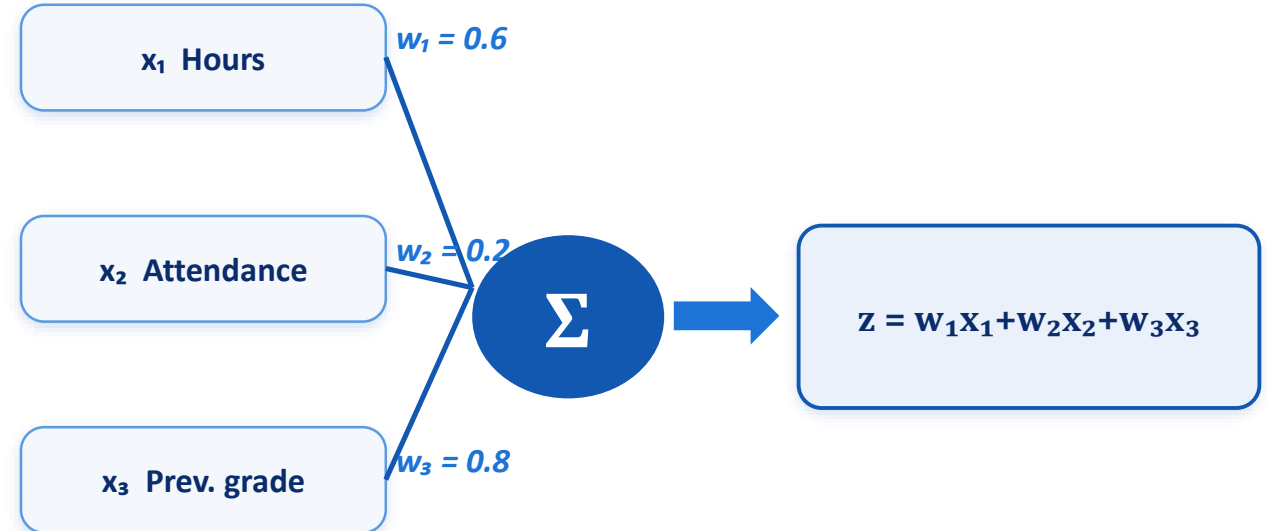
Previous grade 40

Result may differ

Same hours studied — different outcome. The model must learn how important each feature is.

What Is a Weight?

x_1 = Hours studied w_1 = its importance
 x_2 = Attendance w_2 = its importance
 x_3 = Previous grade w_3 = its importance



Important clarification

A weight is not simply “importance.” It determines how strongly an input contributes to the neuron's output — including its direction. Weights carry both magnitude and direction.

Weights Have Magnitude AND Direction

$$w = +2$$

Positive direction

Increasing the input increases the weighted sum.
The feature pushes the decision upward.

$$w = -2$$

Negative direction

Increasing the input decreases the weighted sum.
The feature pushes the decision downward.

So a weight is a learnable scaling factor with a sign — it decides how much, and in which direction, an input matters.

Why Do We Multiply? $w \times x$

We multiply so the model can control how much each input contributes. Same input — completely different contribution.

$$x_1 = 10, \quad w_1 = 2$$

$$w_1 x_1 = 20$$

$$x_1 = 10, \quad w_1 = 0.1$$

$$w_1 x_1 = 1$$

The weight acts as a learnable scaling factor on every input.

What Is the Bias?

What if the neuron needs to shift its output even when all inputs are zero? That's what the bias term b provides.

$$Z = W_1X_1 + W_2X_2 + W_3X_3$$

Without bias — forced through the origin

$$Z = W_1X_1 + W_2X_2 + W_3X_3 + b$$

With bias — free to shift

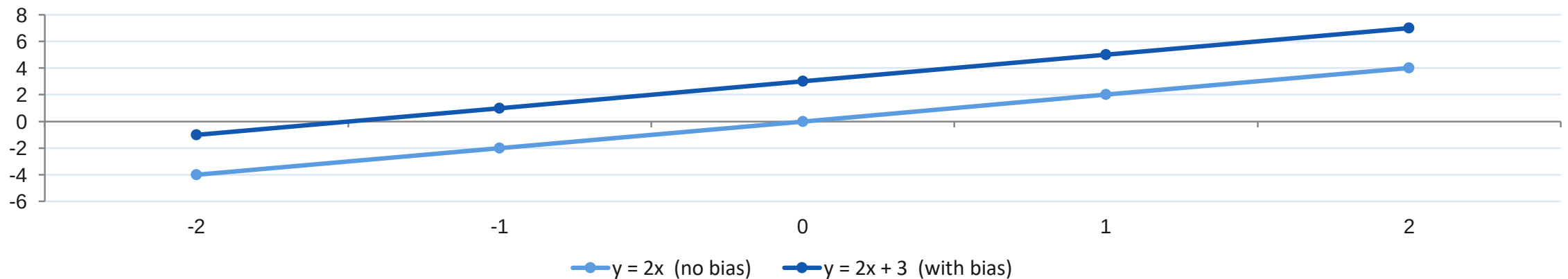
Familiar analogy: $y = mx + b$

m controls the slope

b controls the vertical shift

$z = wx + b$ works the same way for a neuron.

How bias shifts the line



The Weighted Sum, Step by Step

- | | | | |
|---|-------------|--------------|-------------------------------------|
| 1 | $x_1 = 5$ | $w_1 = 0.5$ | $\rightarrow 0.5 \times 5 = 2.50$ |
| 2 | $x_2 = 0.8$ | $w_2 = 0.2$ | $\rightarrow 0.2 \times 0.8 = 0.16$ |
| 3 | $x_3 = 70$ | $w_3 = 0.05$ | $\rightarrow 0.05 \times 70 = 3.50$ |

plus bias $b = -3$

$$z = 2.50 + 0.16 + 3.50 - 3 = 3.16$$

This number is the weighted sum, also called the pre-activation.

Biological Neuron → Artificial Neuron

Biological neuron

- 1 Dendrites receive signals
- 2 Cell body processes them
- 3 Axon sends the output

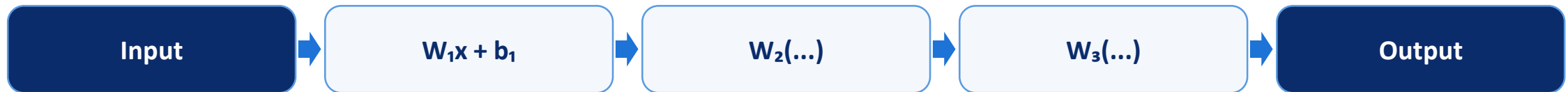
An artificial neuron is inspired by this idea — it is not a realistic simulation of it.

Artificial neuron

- 1 Inputs × weights
- 2 Weighted sum + bias
- 3 Activation function
- 4 Output

Why Do We Need Activation?

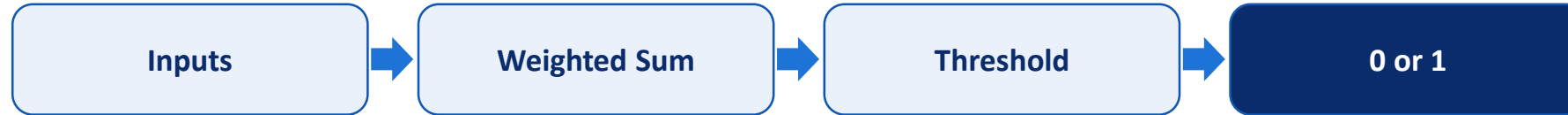
Without activation, every neuron only computes $z = Wx + b$. Stack many linear layers and the whole network still collapses into one linear transformation.



Mathematically, this collapses to one linear transformation — no real depth gained.

Stacking linear layers without a nonlinear activation does NOT give us a truly nonlinear neural network.

THE PERCEPTRON- The Earliest Neuron-Based Model



Decision rule (step function)

$$\hat{y} = 1 \quad \text{if } z \geq 0$$

$$\hat{y} = 0 \quad \text{if } z < 0$$

Quick check

$$z = 2.5 \geq 0 \rightarrow \text{prediction} = 1$$

$$z = -1.2 < 0 \rightarrow \text{prediction} = 0$$

For two inputs: $z = w_1x_1 + w_2x_2 + b$, then the same threshold rule applies.

Perceptron: Pass or Fail?

$$z = 0.7x_1 + 0.4x_2 - 5$$

x_1 = Hours studied

x_2 = Attendance

Given: Hours = 8, Attendance = 90

$$\begin{aligned} z &= (0.7)(8) + (0.4)(90) - 5 \\ &= 5.6 + 36 - 5 = 36.6 \end{aligned}$$

$$36.6 \geq 0$$



PASS

What Is the Decision Boundary?

The decision changes exactly where $z = 0$: $w_1x_1 + w_2x_2 + b = 0$

2 features

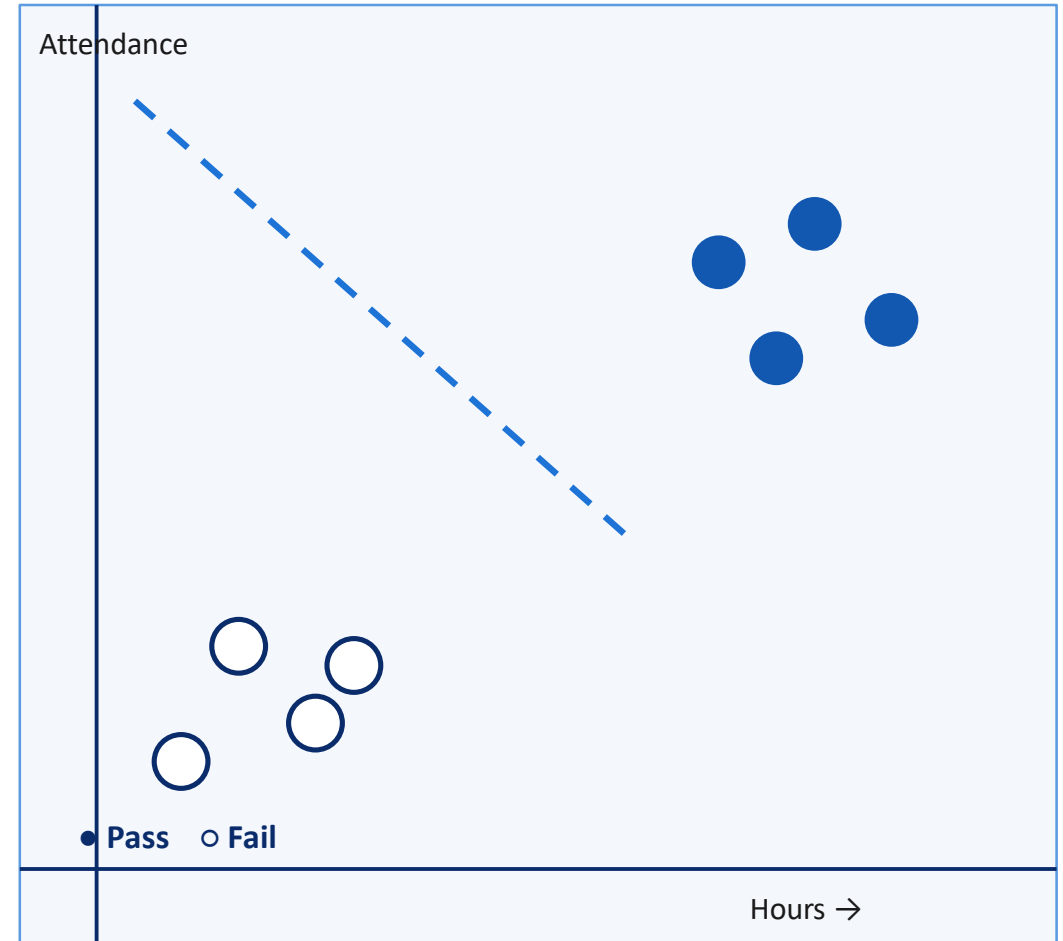
→ a line

3 features

→ a plane

More features

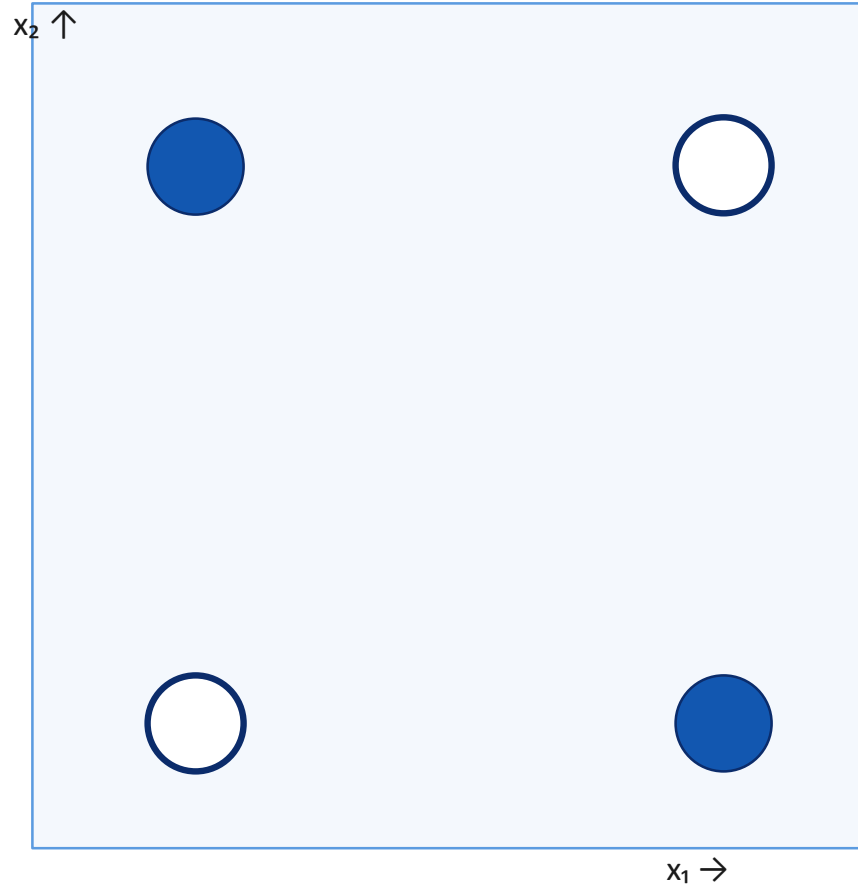
→ a hyperplane



A Single Perceptron Cannot Solve XOR

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

No single straight line can separate the two classes.



Classes sit diagonally — the pattern is not linearly separable.

This was a major limitation of single-layer perceptrons.

From One Neuron to a Neural Network

1

One neuron

$x_1, x_2, x_3 \rightarrow \text{Neuron} \rightarrow \text{Output}$

2

Multiple neurons

Same inputs $\rightarrow$ several neurons in parallel

3

Hidden layer

Different neurons learn different patterns

4

Neural network

Input layer $\rightarrow$ Hidden layer(s) $\rightarrow$ Output layer

Different neurons can specialize: Neuron 1 $\rightarrow$ detects pattern A, Neuron 2 $\rightarrow$ detects pattern B, Neuron 3 $\rightarrow$ combines them, Output $\rightarrow$ final decision.

Representation Learning

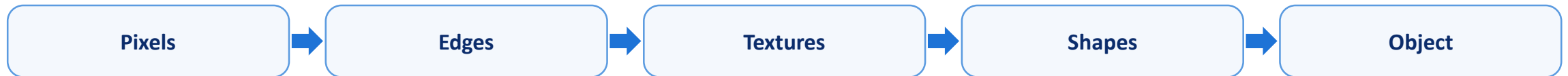
Traditional ML

1. Raw data
2. Human-designed features
3. ML model
4. Prediction

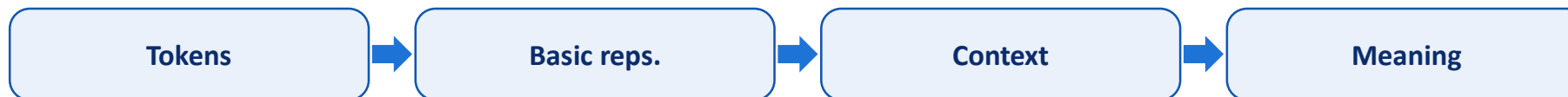
Deep Learning

1. Raw data
2. Layer 1 → simple patterns
3. Layer 2 → combines patterns
4. Layer 3 → higher-level patterns → Prediction

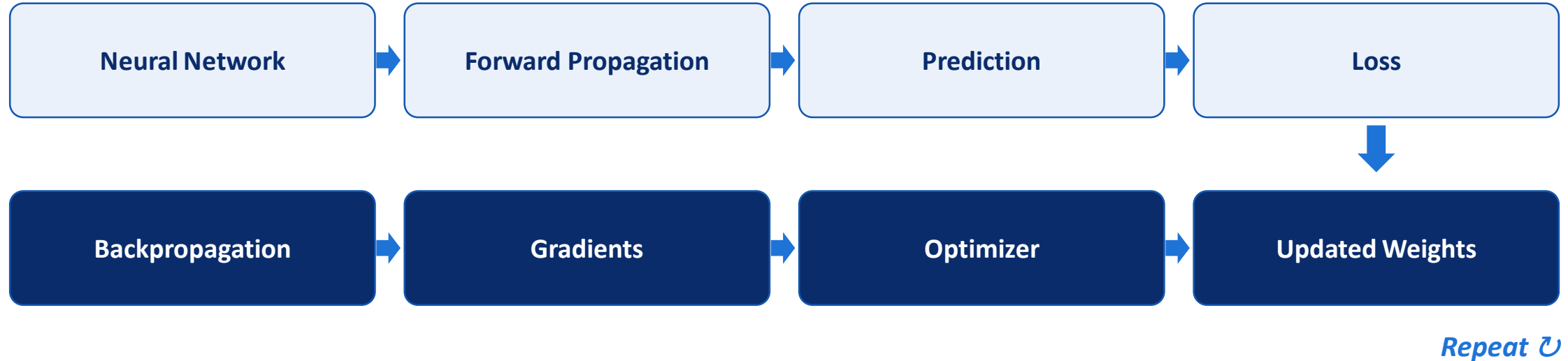
Images:



Text (NLP):



The Training Loop



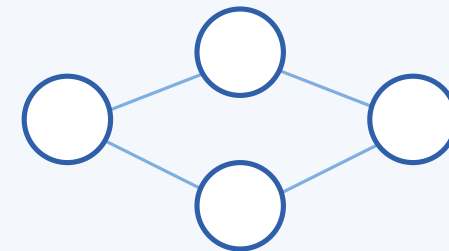
Predict → Measure error → Calculate gradients → Update parameters → Repeat.

Don't Oversimplify It

“A neural network is just many perceptrons.”

That's an oversimplification.

A neural network is a parameterized function composed of layers of transformations — linear transformations plus nonlinear activations — whose parameters are learned from data.



DEEP LEARNING FUNDAMENTALS • PART 2

From One Neuron to a Neural Network

Understanding Forward Propagation, Layer by Layer

Neurons

Layers

Weights & Bias

Activation

Forward Pass

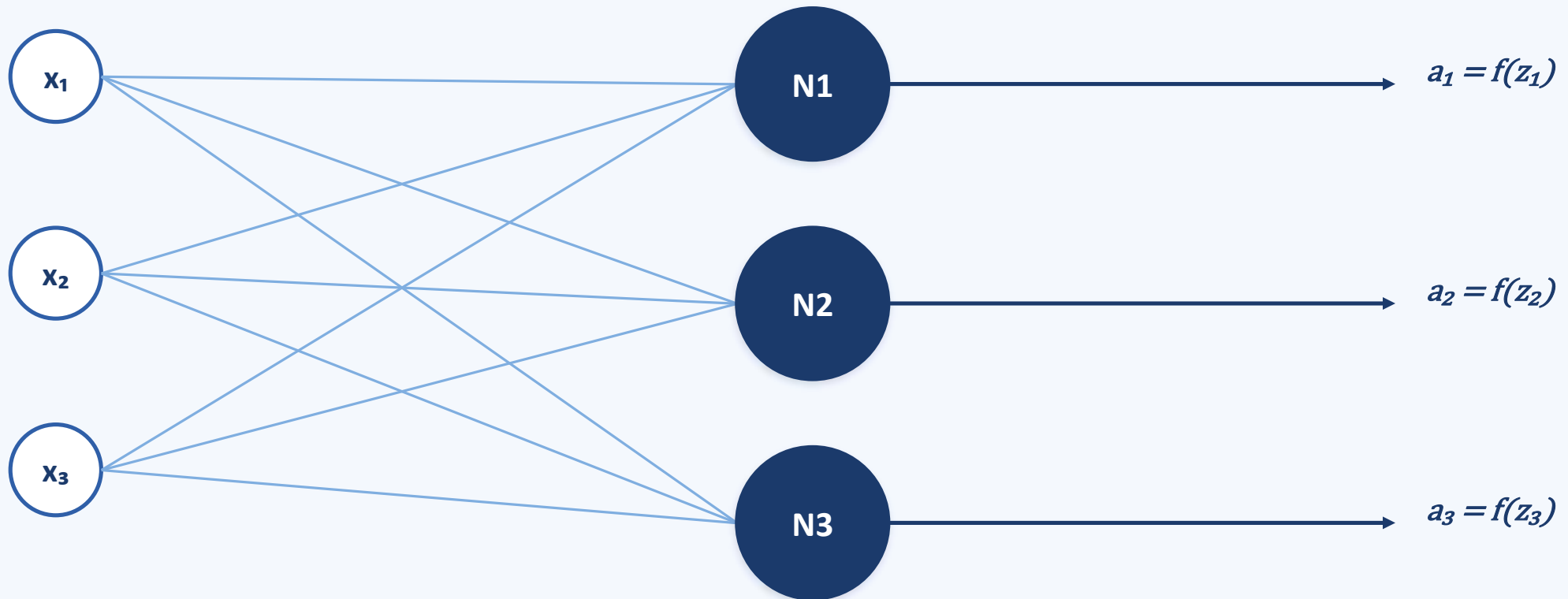
Multiple Neurons, Same Input

Each neuron receives the same three inputs — but has its own weights and bias, so each one learns a different combination of the inputs.

Same 3 inputs

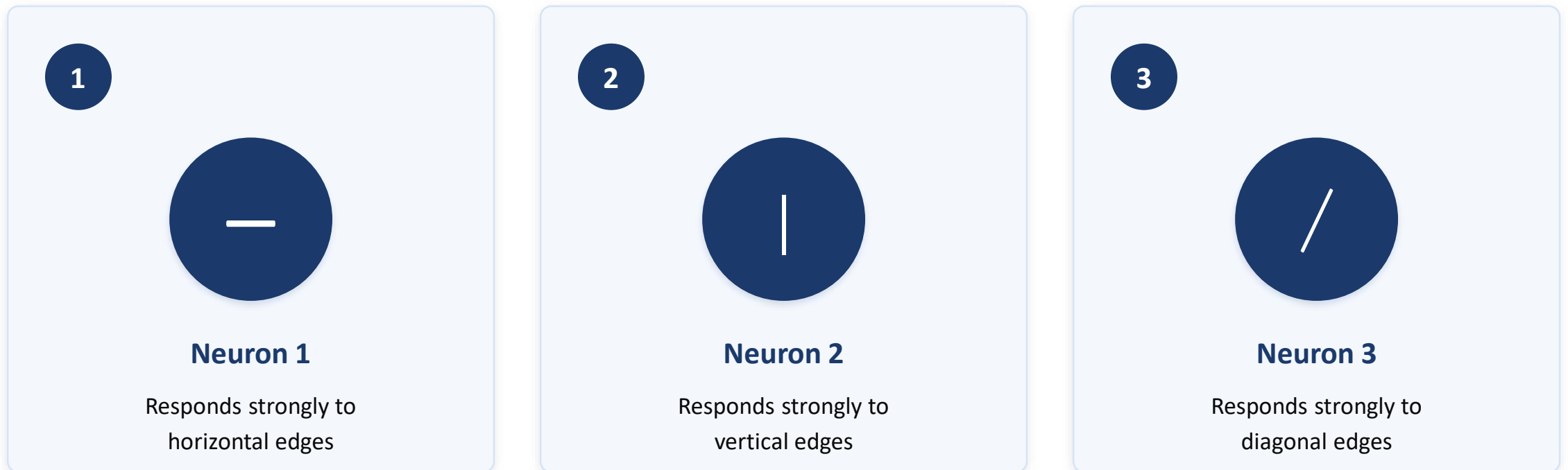
Each neuron: own w , b

Different outputs



Why Do We Need Multiple Neurons?

Imagine we're processing an image. Different neurons can specialize in detecting different patterns — all from the very same input.

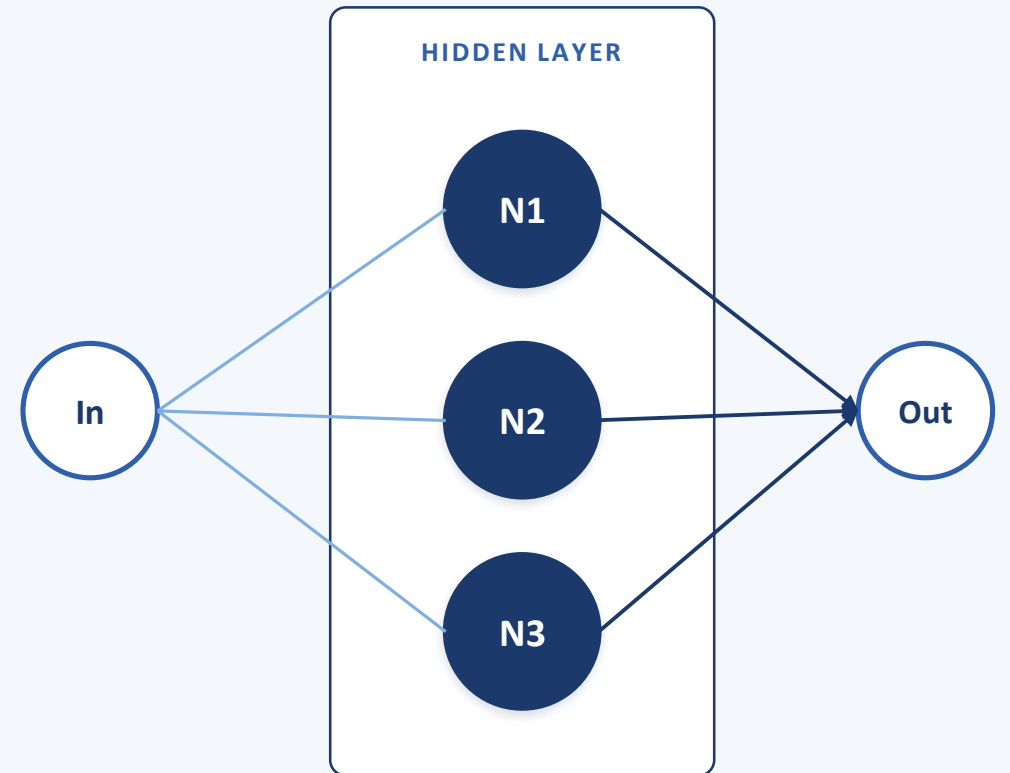


Same input, multiple neurons → the network learns several patterns at once, not just one.

A Layer Is a Group of Neurons

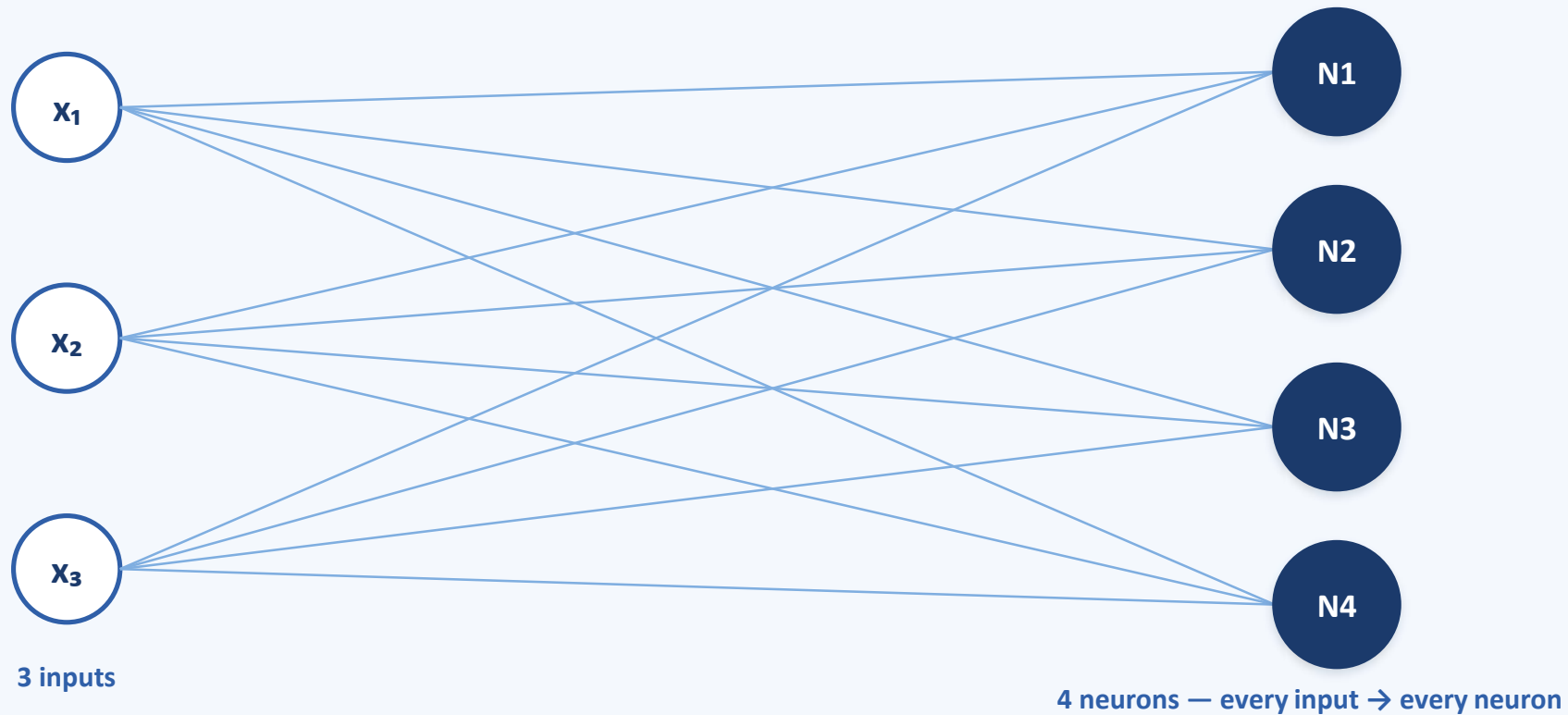
A layer is a collection of neurons that process the same input and produce a set of outputs together.

When several neurons work side by side on the same inputs, that group is called a layer. Stack layers together, and you get a neural network.



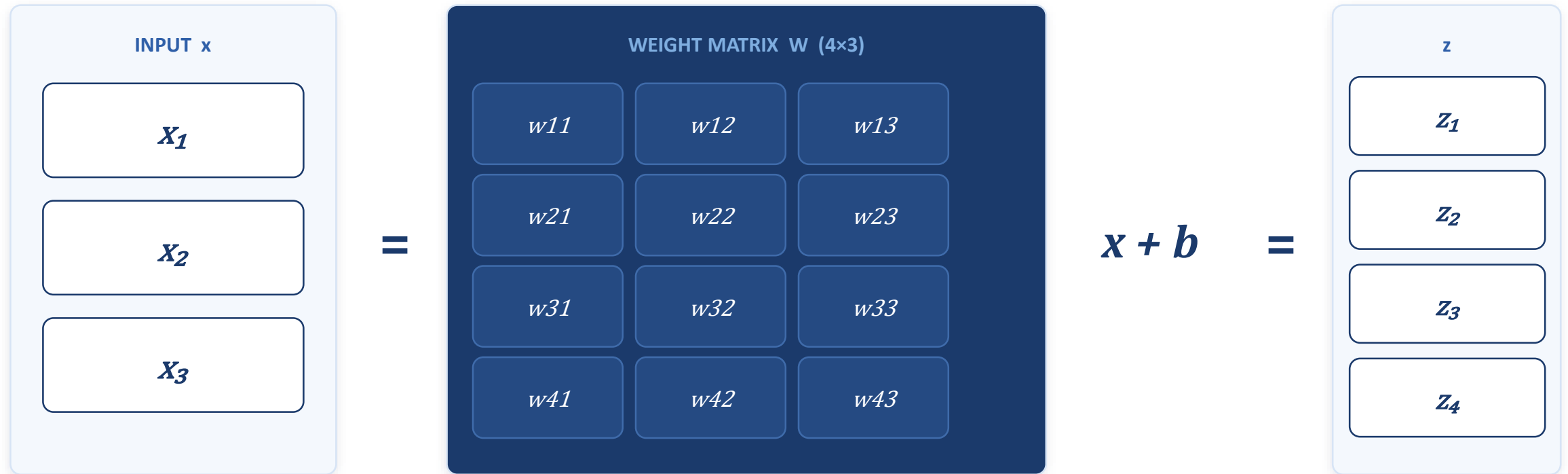
The Fully Connected (Dense) Layer

With 3 inputs and 4 neurons, every input connects to every neuron — that's what makes a layer “fully connected,” or dense.



Writing It All as $z = Wx + b$

Instead of calculating every neuron's equation one at a time, we collect all the weights into a matrix W and compute the whole layer in one step.

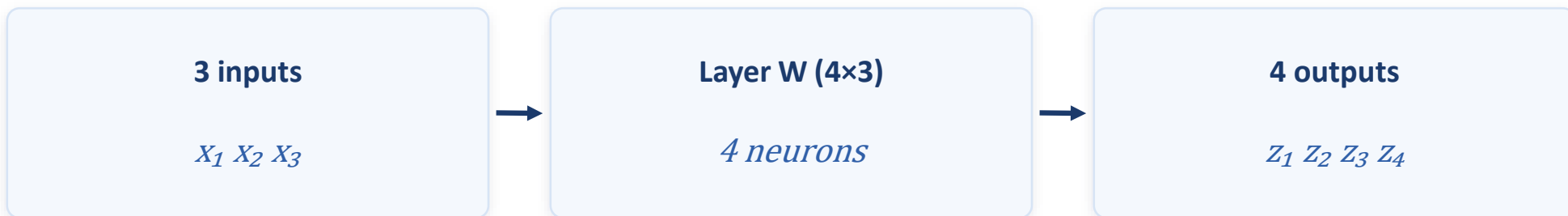


What Do the Matrix Dimensions Mean?

With 3 input features and 4 neurons in the layer, the weight matrix W must be shaped 4×3 — one row per neuron, one column per input.

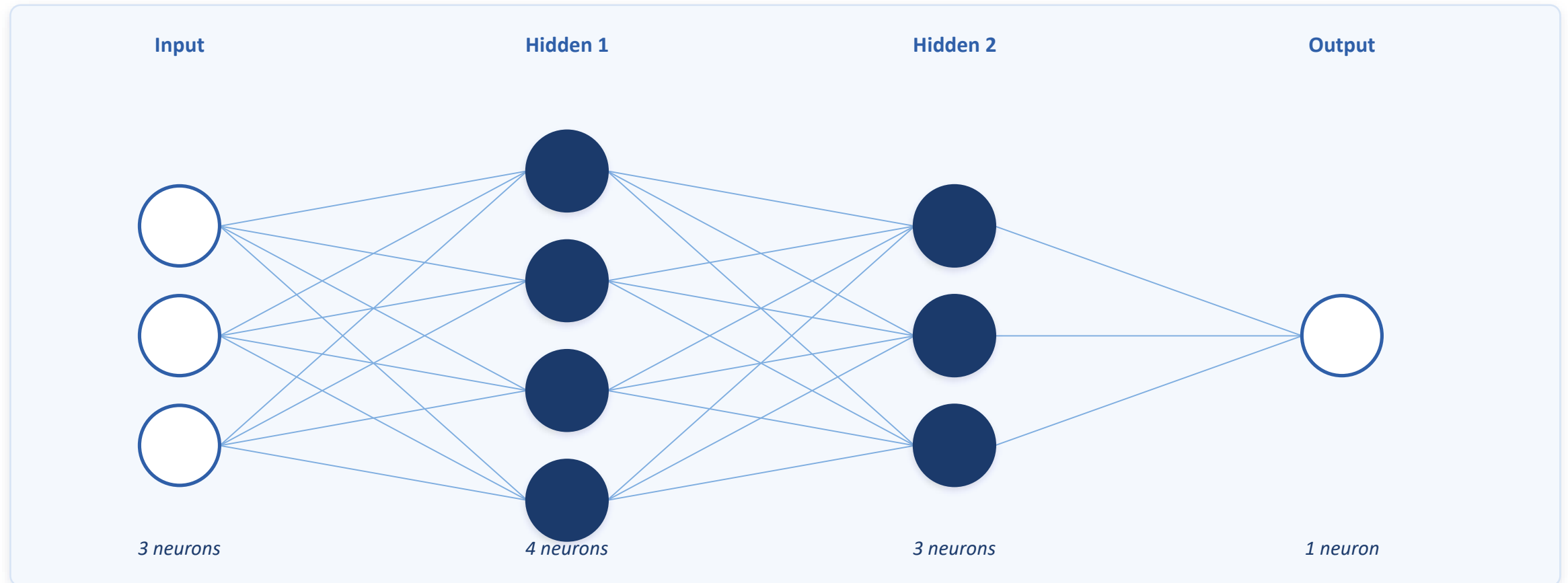
A diagram illustrating matrix multiplication. On the left, a dark blue square contains the letter W in white, with the dimensions (4×3) in white below it. To its right is a light blue square containing the letter X in dark blue, with the dimensions (3×1) in dark blue below it. An equals sign $=$ is placed between these two squares and a third light blue square on the right. This third square contains the letter Z in dark blue, with the dimensions (4×1) in dark blue below it.

The inner dimensions (3 and 3) must match — and cancel out, leaving a 4×1 result: one z value per neuron.



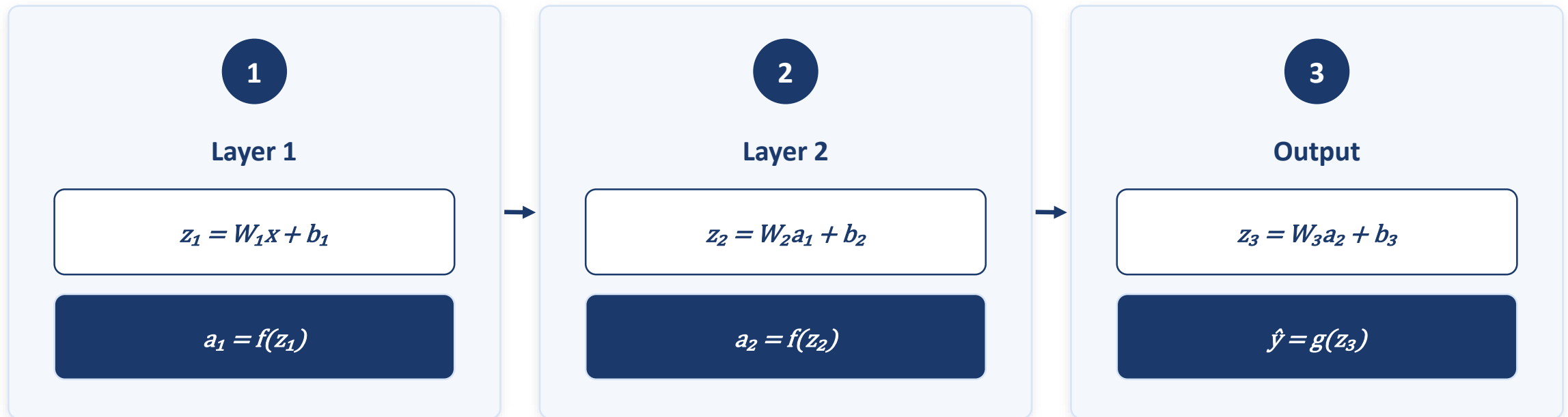
From One Layer to Many

Stack layers back to back, and information flows from the input, through each hidden layer, to a final output.

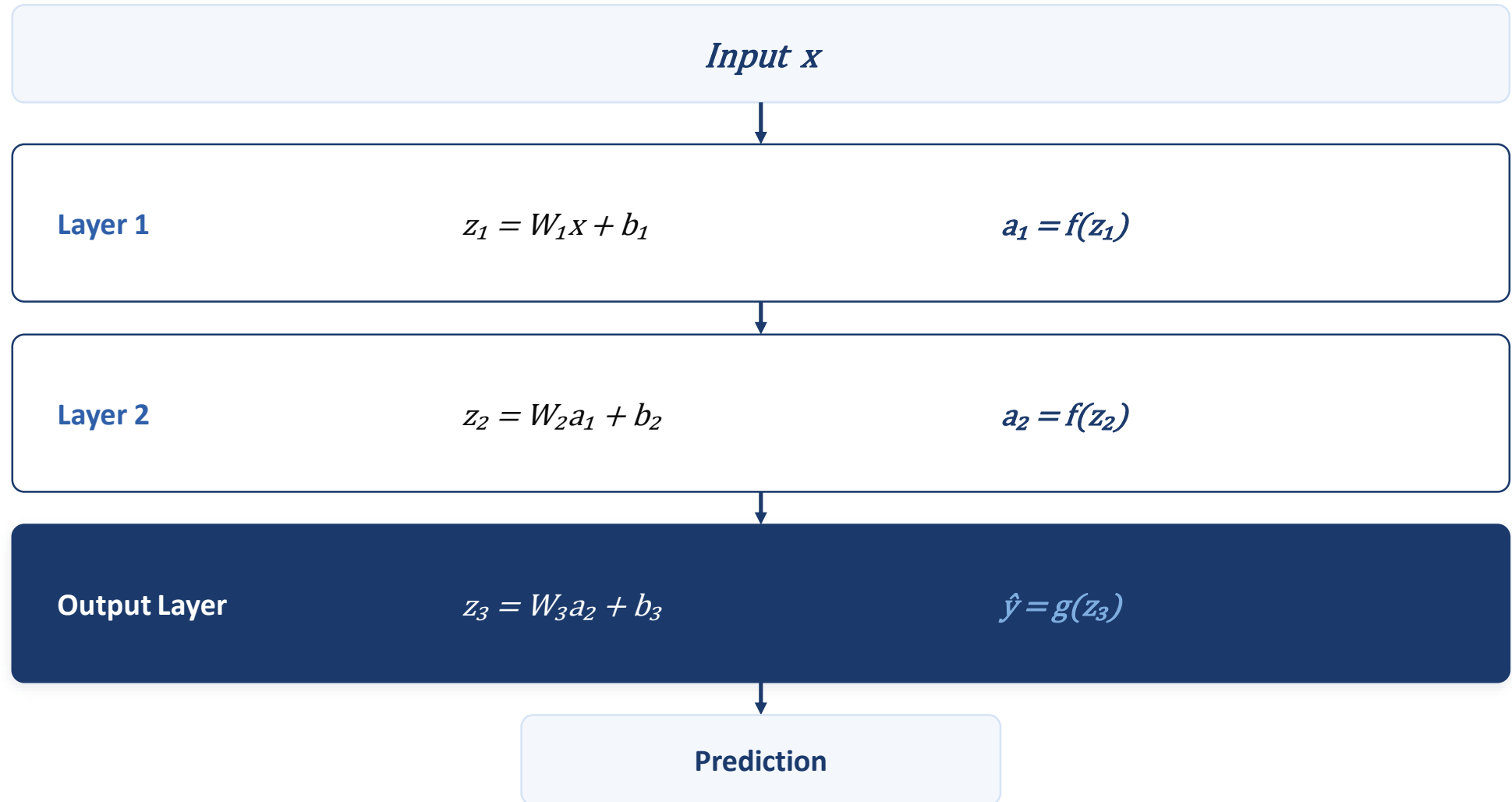


Forward Propagation, Defined

Forward propagation is the process of passing the input through every layer of the network — applying the learned weights, biases, and activation functions — to produce a final prediction.

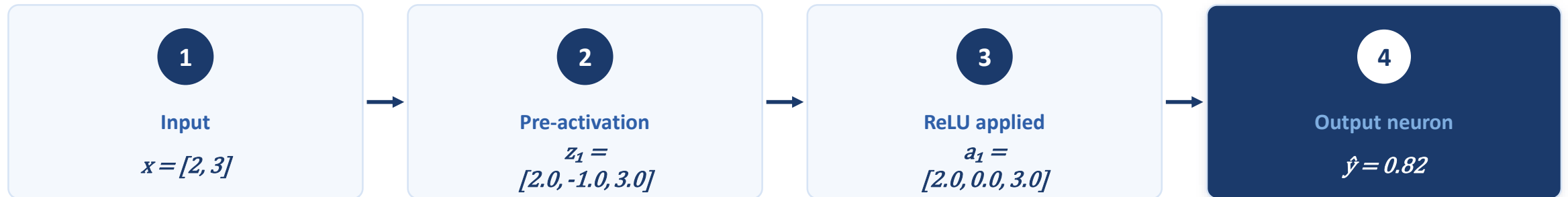


The Complete Forward Pass



Let's Walk Through a Simple Example

Input $x = [2, 3]$, one hidden layer with 3 neurons, then a single output neuron.



What does 0.82 mean?

With a sigmoid output, 0.82 is read as a probability — about an 82% chance the input belongs to class 1. Applying a 0.5 threshold:

$$0.82 \geq 0.5$$



Class 1

Output Activation Depends on the Task

Not every network ends the same way. The right output activation depends on what you're predicting.

Binary Classification

Sigmoid

Examples

Pass / Fail

Cat / Not Cat

Spam / Not Spam

0.82 → 82% "class 1"

Multi-Class Classification

Softmax

Examples

Cat → 0.70

Dog → 0.20

Horse → 0.10

Probabilities sum to 1

Regression

Linear (none)

Examples

House price

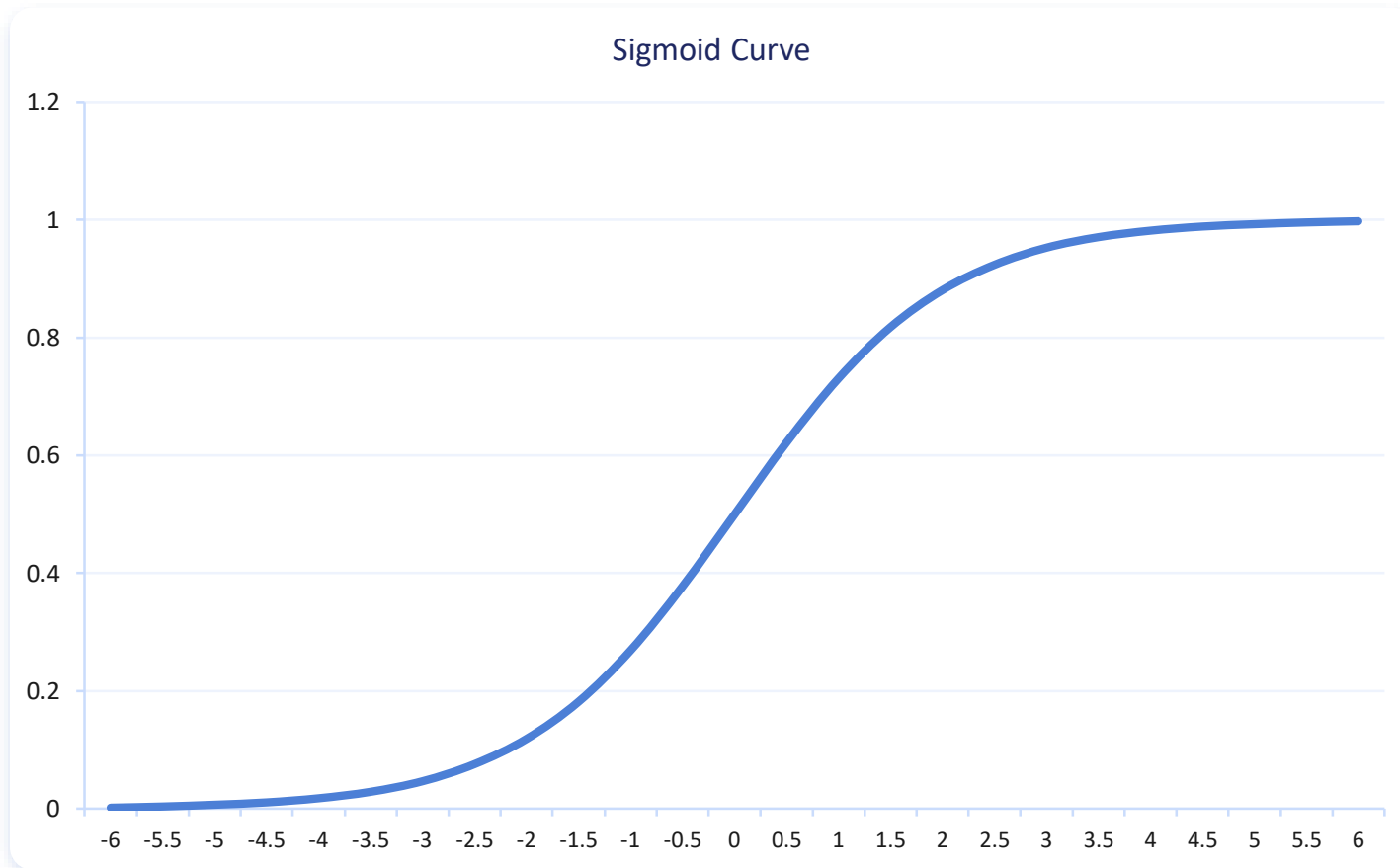
≈ \$2,500,000

Not squeezed into 0–1

Sigmoid — Squashing to (0, 1)

$$\sigma(x) = 1 / (1 + e^{-x})$$

Output range: $0 < \sigma(x) < 1$



Main Use: Binary Classification Output

Neural Network → Sigmoid → 0.87

0.87 = 87% probability of class 1

Problem

Can suffer from vanishing gradients, especially for very large positive or negative inputs — the curve flattens out at both ends.

Tanh — Zero-Centered Squashing

$$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

Output range: $-1 < \tanh(x) < 1$



Main Use

Historically important, still used in RNNs, LSTMs, and some specialized architectures.

Advantage over Sigmoid — zero-centered:

Sigmoid: 0 to 1 Tanh: -1 to 1

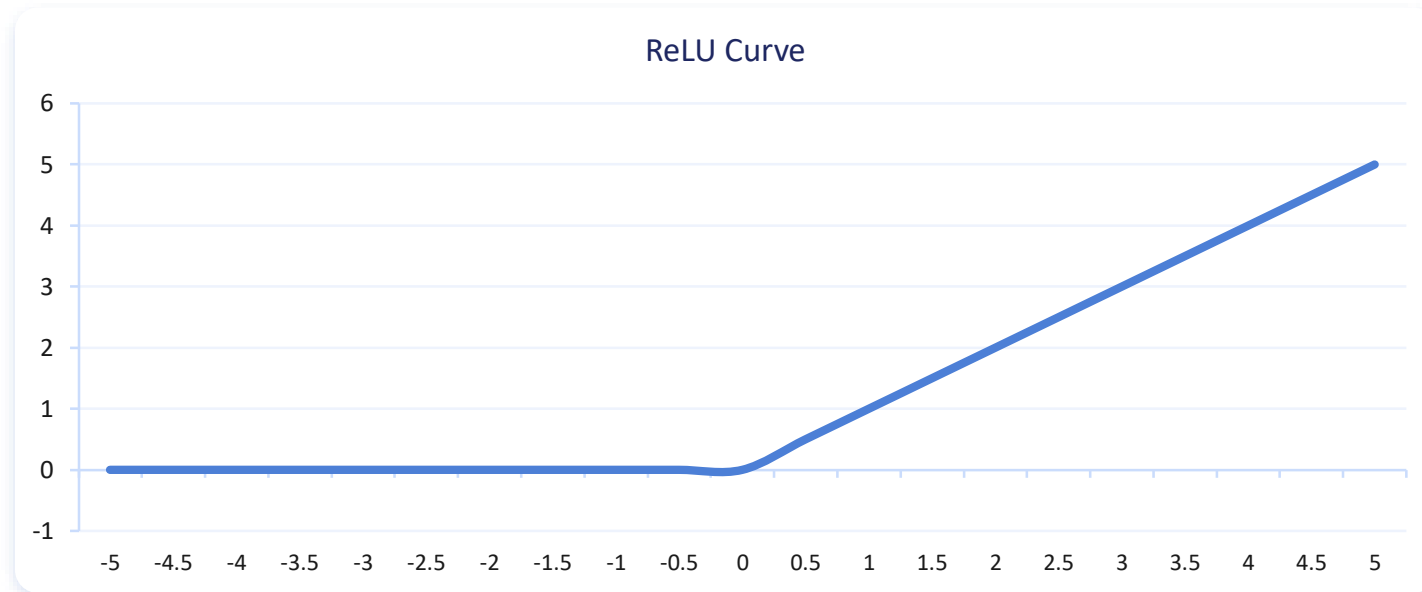
Problem

Like Sigmoid, it can also suffer from vanishing gradients at the extremes.

ReLU — The Default for Hidden Layers

$$\text{ReLU}(x) = \max(0, x)$$

$$x < 0 \rightarrow 0 \quad x > 0 \rightarrow x$$



Why Is ReLU Popular?

Simple and computationally cheap. Helps deep networks train more effectively than sigmoid/tanh in many architectures.

Main Use: Hidden Layers

Linear → **ReLU** → **Linear** → **ReLU** → ...

Stacked through the hidden layers of a deep network.

ReLU(-5)

0

ReLU(-2)

0

ReLU(0)

0

ReLU(3)

3

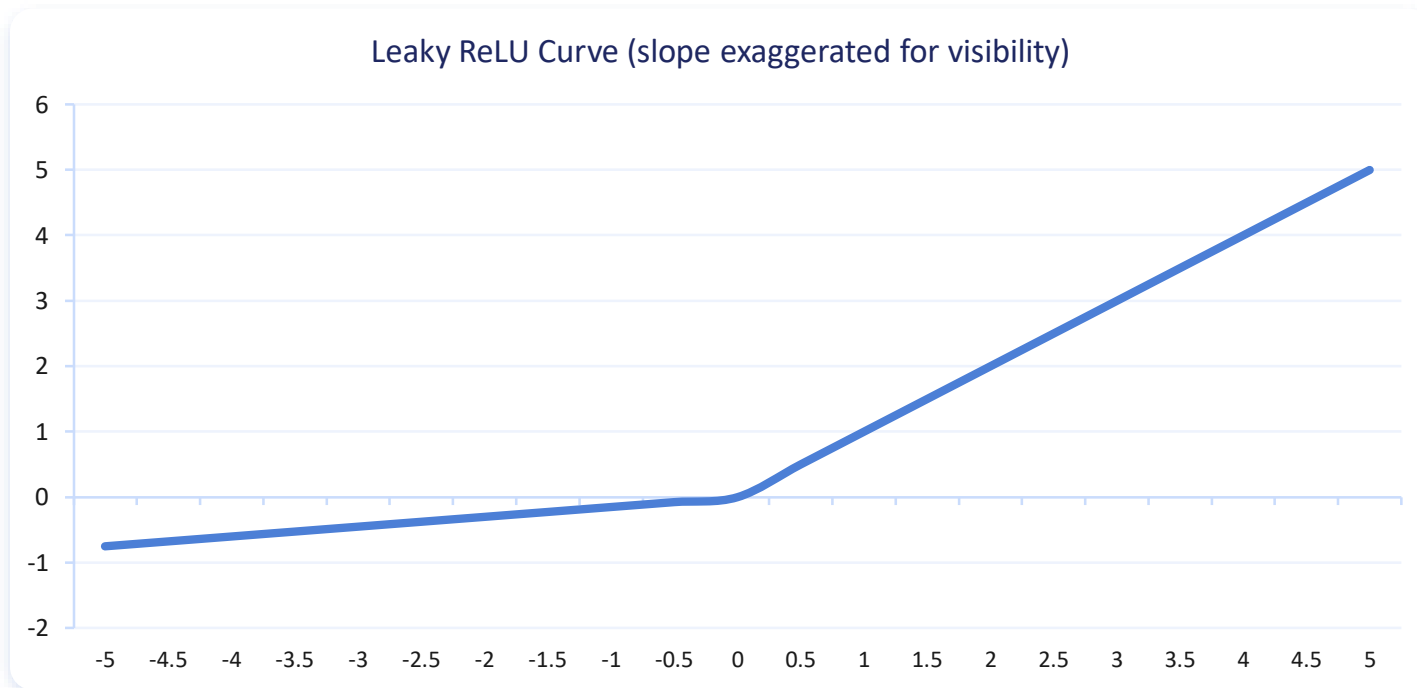
ReLU(7)

7

Leaky ReLU — Fixing the Dying ReLU Problem

The Problem With ReLU: negative input $\rightarrow 0 \rightarrow$ a neuron can get stuck always producing zero (“dying ReLU”).

$$f(x) = x \text{ if } x > 0, \quad \alpha x \text{ if } x \leq 0 \quad (\text{typically } \alpha = 0.01)$$



Worked Example ($\alpha = 0.01$)

LeakyReLU(-5) = -0.05

LeakyReLU(5) = 5

Main Use

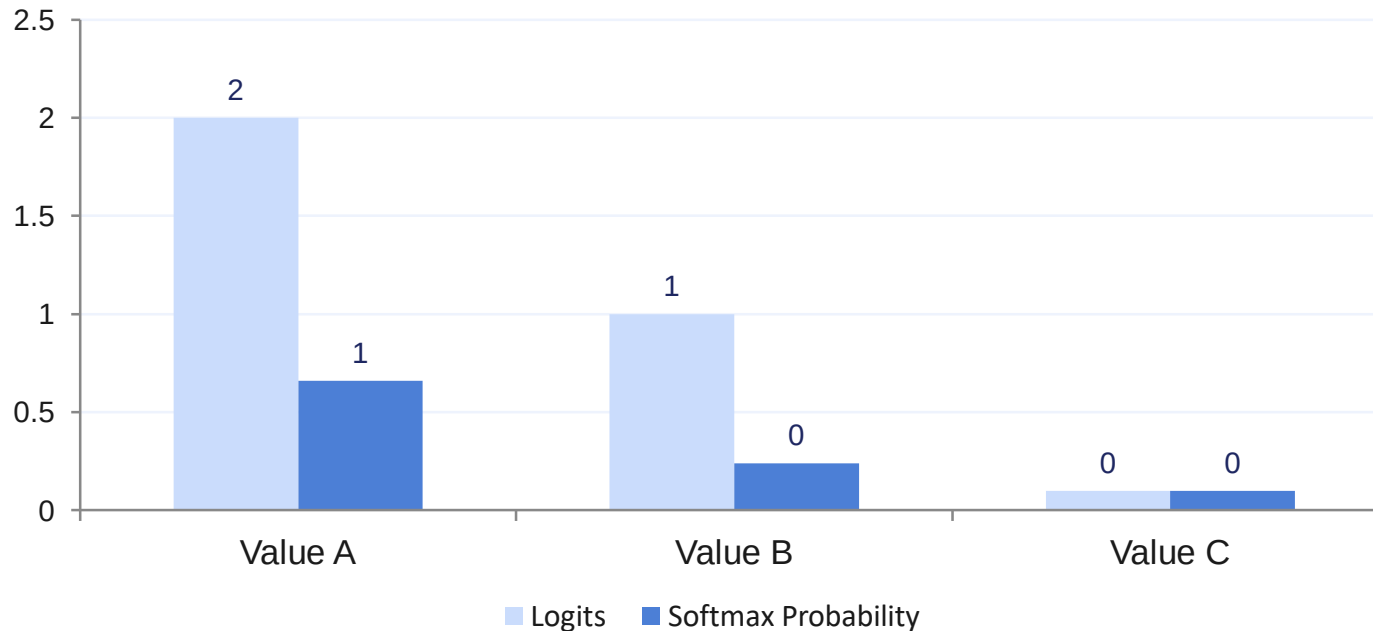
Hidden layers, when you want to reduce the risk of dying ReLU neurons.

Softmax — Logits to Probabilities

$$\text{Softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$$

Converts a vector of logits into probabilities that sum to 1

Example: Logits → Softmax



$0.66 + 0.24 + 0.10 = 1.00$ — the three outputs always sum to exactly 1.

Main Use: Multi-Class Output

Image → Network → [Cat, Dog, Horse] → Softmax
→ [0.05, 0.90, 0.05]

Prediction: Dog (highest probability, 0.90)

Why Softmax

One probability per class, always summing to 1 — used only in the output layer for multi-class classification.

Choosing an Activation Function

Function	Range	Typical Use	Watch Out For
Sigmoid	$(0, 1)$	Binary output layer	Vanishing gradients
Tanh	$(-1, 1)$	RNNs / LSTMs	Vanishing gradients
ReLU	$[0, \infty)$	Hidden layers (default)	Dying ReLU
Leaky ReLU	$(-\infty, \infty)$	Hidden layers	Extra hyperparameter α
Softmax	$(0, 1)$, sums to 1	Multi-class output layer	Only for the output layer

Rule of thumb: ReLU (or Leaky ReLU) for hidden layers · Sigmoid for binary output · Softmax for multi-class output.

HOW NEURAL NETWORKS LEARN

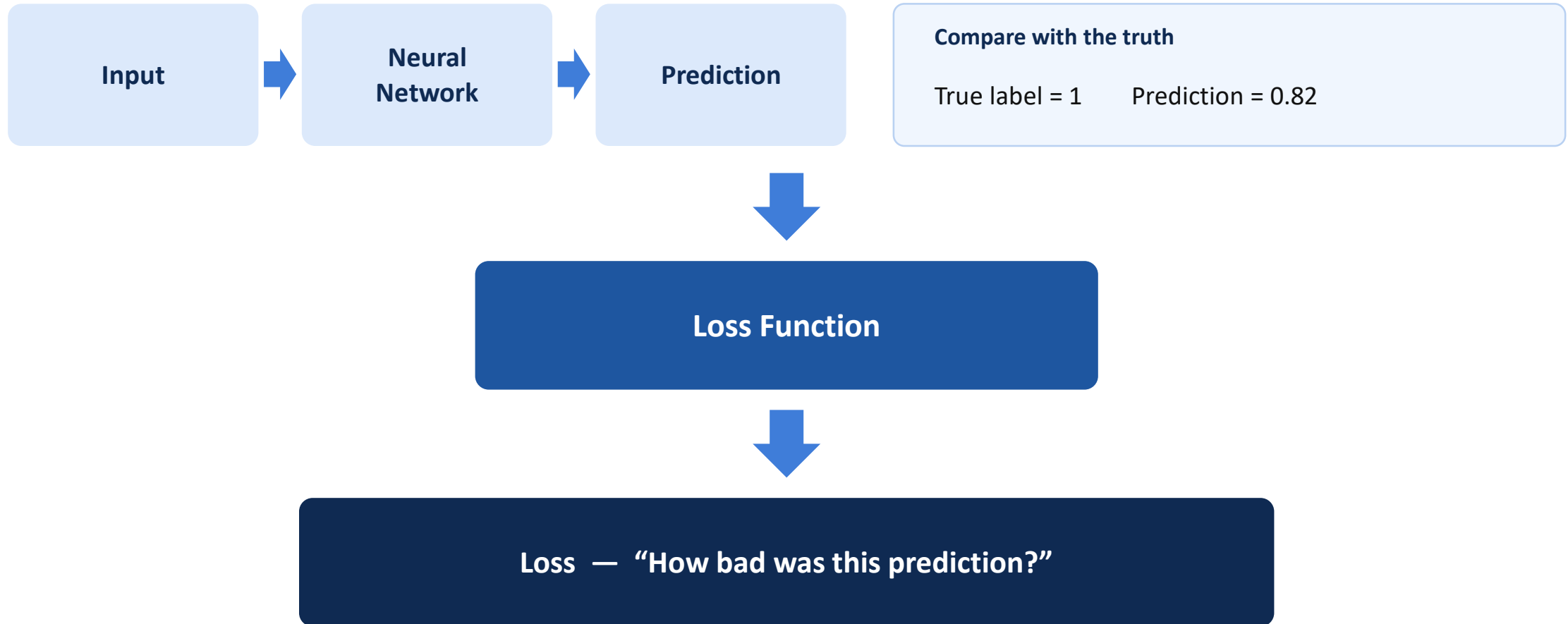
Loss, Gradients & Backpropagation

From a single prediction to a full training loop — how a network measures its mistakes and learns to correct them.



How a Prediction Becomes a Loss

FORWARD PASS

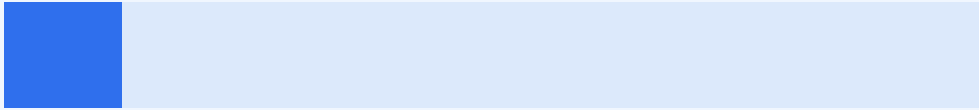


The loss function takes the true value and the prediction, and returns a single number describing how undesirable the prediction is.

Small Loss = Good. Large Loss = Bad.

The loss is a number representing how undesirable a prediction is, according to the objective we chose to optimize.

SMALL LOSS



Good prediction — model is close to the truth

LARGE LOSS



Bad prediction — model is far from the truth

THE LOSS PIPELINE

True value + Prediction



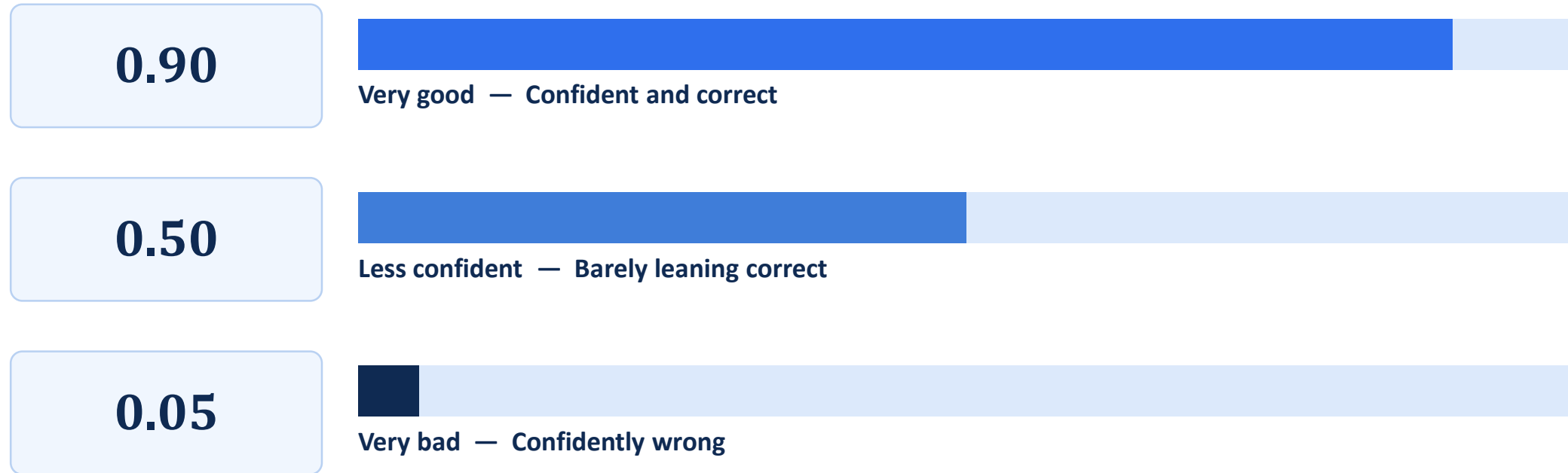
Loss Function



Loss

Loss Reflects How Confident the Model Is

True label = 1 — three different predictions for the same example:



A good loss function (like cross-entropy) captures this difference — accuracy alone cannot.

Why Not Just Use Accuracy?

Accuracy is not a good training signal — it can't tell the optimizer how to adjust the weights.

Two predictions for the same positive example, threshold = 0.5:

Prediction A

0.51

Barely over the line

Correct

Prediction B

0.99

Confidently right

Correct

ACCURACY SEES BOTH AS SIMPLY “CORRECT”

But 0.99 is far more confident than 0.51. A loss function such as cross-entropy preserves that difference, which is exactly the information the optimizer needs during training.

Different Problems Need Different Losses

Regression

House price · Temperature · Salary

Mean Squared Error

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

Binary Classification

Pass / Fail · Spam / Not spam · Cat / Not cat

Binary Cross-Entropy

$$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Multi-Class Classification

Cat · Dog · Horse

Categorical Cross-Entropy

with Softmax outputs over all classes

Loss vs. Metric

Loss

Used to guide training

Prediction



Loss



Backpropagation



Gradients

Metric

Used to evaluate performance

Accuracy



Precision



Recall



F1 Score

The loss tells the optimizer what to minimize; evaluation metrics tell us how well the model performs from our chosen perspective.

Why Isn't Loss Enough?

Loss = 2.5

We know the model is performing poorly — but not which weights caused it.

A network may have thousands or millions of parameters:

w_1

w_2

w_3

w_4

...

$w_{1000000}$

For each weight, two questions need answering:

1. Should this weight change, and by how much?
2. Should it increase or decrease?

The loss alone can't answer this. We need gradients.

Forward Pass, Backward Pass

Backpropagation calculates the gradients of the loss with respect to the network's parameters.

FORWARD — computes values



BACKWARD — computes gradients



The forward pass calculates the values. The backward pass calculates the gradients — the gradients are propagated backward through the computational graph, which is why it's called “backpropagation.”

The Chain Rule

The loss doesn't depend on w directly — it depends on a chain of computations, so we use the chain rule.



The weight affects the loss through this chain of computations.

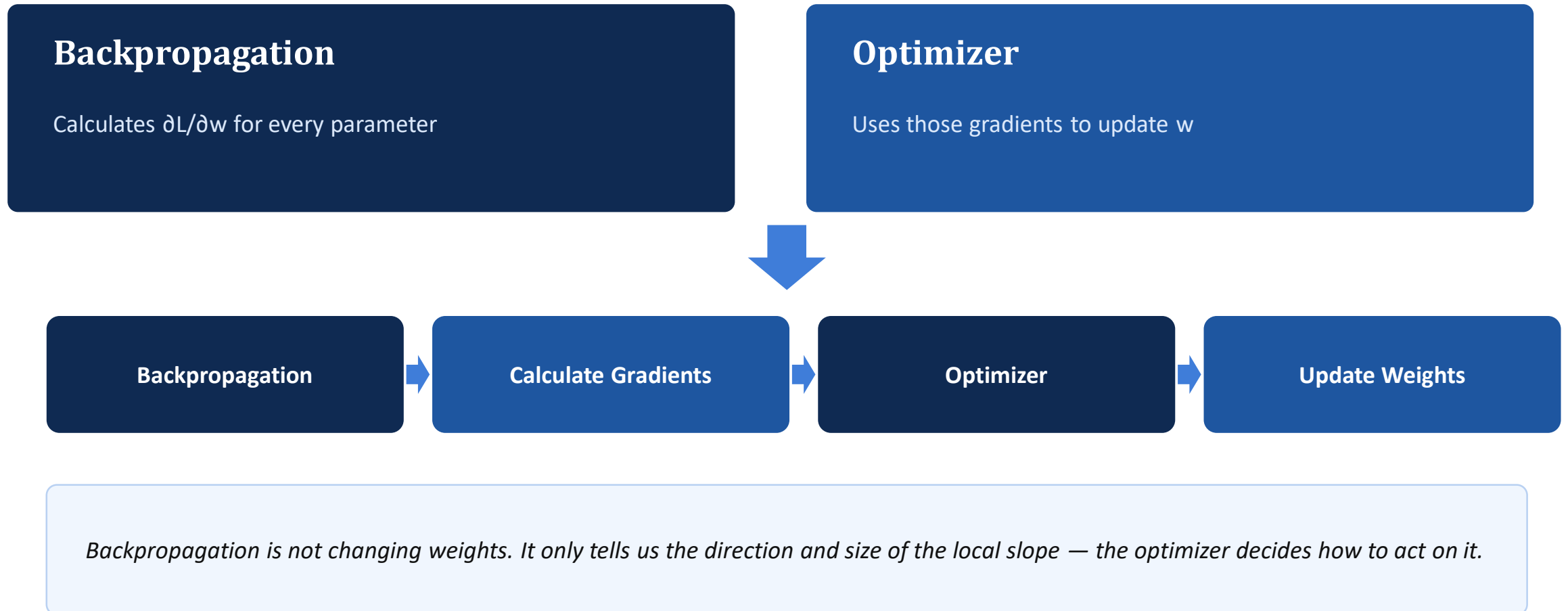
$$\partial L / \partial w = (\partial L / \partial a) \cdot (\partial a / \partial z) \cdot (\partial z / \partial w)$$

We work backward through the computational chain — that's the essence of backpropagation:



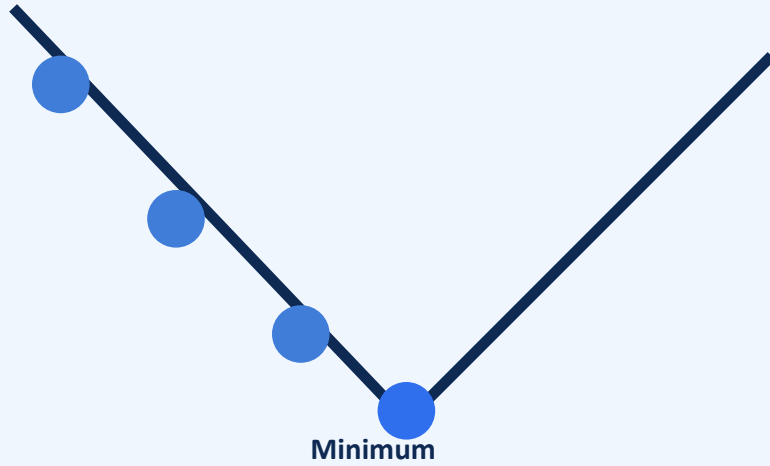
Backpropagation Is Not the Weight Update

Backpropagation calculates the gradients. A separate optimizer uses them to actually change the weights.



Descending Toward the Minimum

LOSS SURFACE



Gradient descent moves opposite the gradient — downhill toward smaller loss.

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \partial L / \partial w$$

w = weight η = learning rate $\partial L / \partial w$ = gradient

NUMERIC EXAMPLE

Current weight	0.50
Learning rate	0.10
Gradient	-2

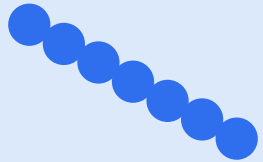
$$w_{\text{new}} = 0.50 - (0.10)(-2) = 0.70$$

What Does the Learning Rate Do?

The learning rate η controls how large each update step is.

Small $\eta = 0.001$

Small, careful steps.
Training can be very slow.



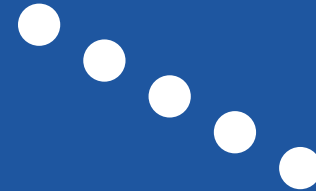
Large $\eta = 1$

Large steps.
Can overshoot or become unstable.



Good η

Balanced steps.
Steady convergence to the minimum.



PART TWO — TUNING THE UPDATE

Learning Rate & Optimizers

How large should each step be, and what strategy should decide it?
From plain gradient descent to Adam and AdamW.



The Learning Rate Controls Step Size

Same gradient. Different learning rate. Very different update.

GRADIENT = 2

$$\eta = 0.1$$

$$\Delta w = -0.1 \times 2 = -0.2$$

Larger update

GRADIENT = 2

$$\eta = 0.01$$

$$\Delta w = -0.01 \times 2 = -0.02$$

Smaller update

Large $\eta \rightarrow$ large parameter updates

Small $\eta \rightarrow$ small parameter updates

The Learning Rate Isn't Learned From Data

Weights & Biases

Learned parameters

Updated automatically during training, based on gradients computed from the data.

Learning Rate

A hyperparameter

Chosen or scheduled by us, before or during training — not derived from gradients.

You don't say “the network learned the learning rate.”

You say “we selected or scheduled the learning rate” — for example, learning rate = 0.001.

Why Do We Need Optimizers?

Basic gradient descent works — but real networks make it hard in practice.

Millions / billions of parameters

+

A complicated loss surface

Different gradient
magnitudes

Noisy
gradients

Plateaus

Steep
directions

Shallow
directions

Difficult
landscapes

So researchers developed better ways to use gradients — that's where optimizers come in.

An optimizer is the algorithm responsible for deciding how to update the model's parameters using the gradients.

Gradient Descent → SGD

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla L$$

GRADIENT DESCENT

Gradient

× Learning rate

Move opposite
the gradient

New weight

STOCHASTIC GRADIENT DESCENT

Use a mini-batch instead of the full dataset for every update.

100,000
samples

Mini-batch
of 32

Calculate
gradient

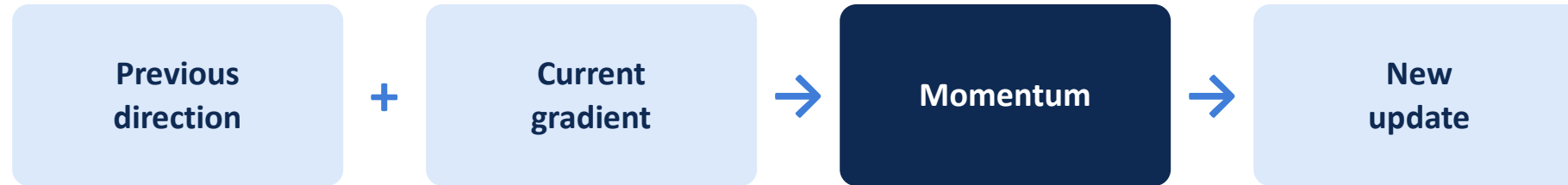
Update
parameters

Calculating the gradient on the full training set for every update is impractical at scale.

In modern deep learning, when people say “SGD,” they usually mean mini-batch SGD.

Momentum

Basic gradient descent only looks at the current gradient. Momentum adds velocity from previous updates.



Instead of “where should I go based only on the current gradient?” momentum asks “where have I been moving, and where is the current gradient telling me to go?”

**Accelerates movement
in consistent directions**

**Reduces
oscillations**

**Helps move through
shallow regions**

AdaGrad

AdaGrad adapts the learning rate separately for each parameter, based on its history of gradients.

Parameter 1

Frequent, large gradients

Smaller future steps

Parameter 2

Rare, small gradients

Relatively larger steps

Useful for sparse features — AdaGrad adapts the learning rate separately for different parameters.

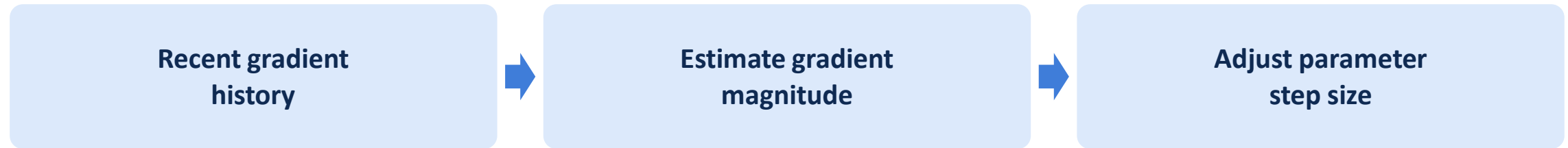
WEAKNESS

Its effective learning rates can shrink more and more during training — which motivates RMSProp.

RMSProp

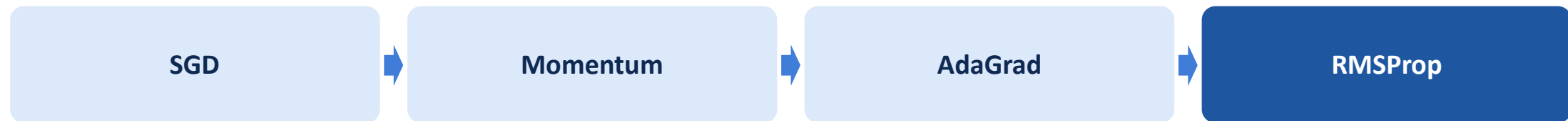
Large gradient $\rightarrow$ small step
Small gradient $\rightarrow$ large step

RMSProp also adapts the learning rate, but keeps a moving average of squared gradients instead of accumulating forever.



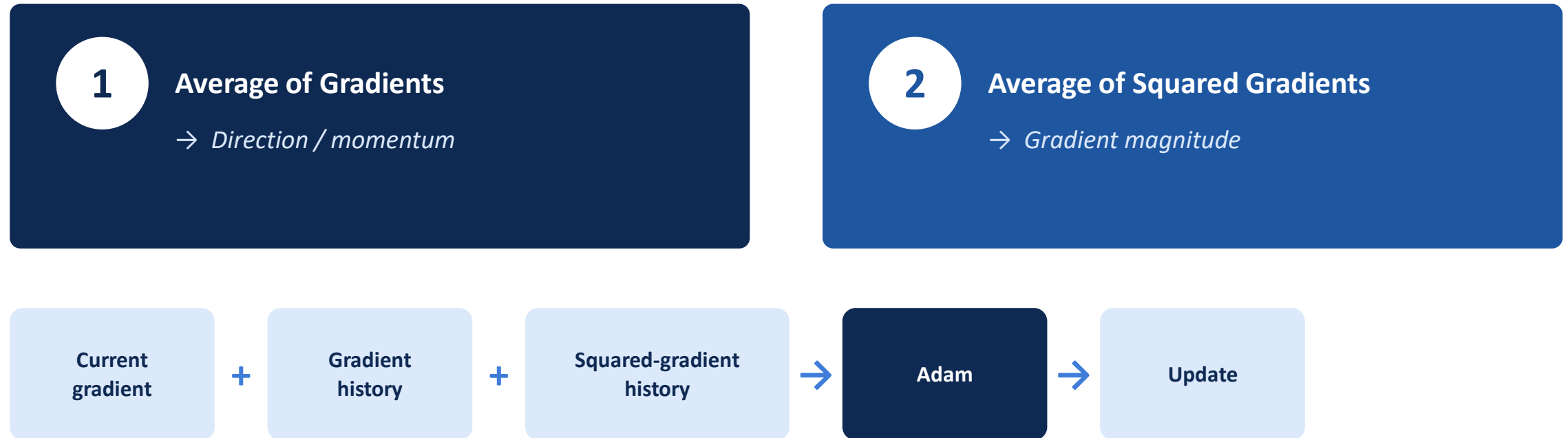
Prevents the learning rate from shrinking as aggressively as AdaGrad.

THE PROGRESSION SO FAR



Adam — Adaptive Moment Estimation

Adam combines momentum with adaptive scaling of gradients, tracking two running quantities.



This is why Adam often works very well as a general-purpose optimizer.

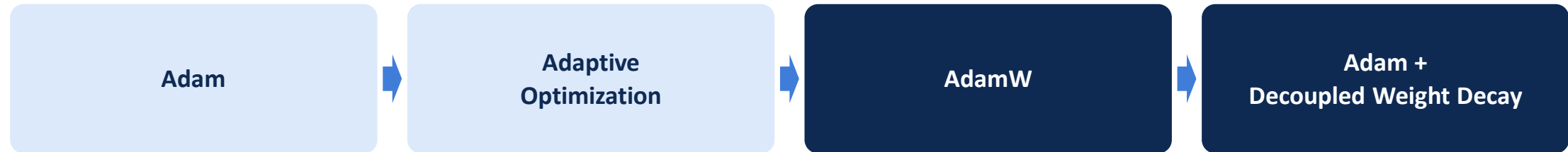
Adam vs. SGD

	SGD	Adam
Basic idea	Gradient-based updates	Adaptive gradient-based updates
Momentum	Optional	Built into Adam's formulation
Per-parameter adaptation	Limited	Yes
Usually easy to get working	Sometimes	Often
Memory	Lower	Higher
Common use	Many DL tasks	Very common default

“Adam is always better” is incorrect — the best optimizer depends on architecture, dataset, learning rate, batch size, regularization, and task.

AdamW

AdamW is Adam with decoupled weight decay — not simply “Adam + W.”

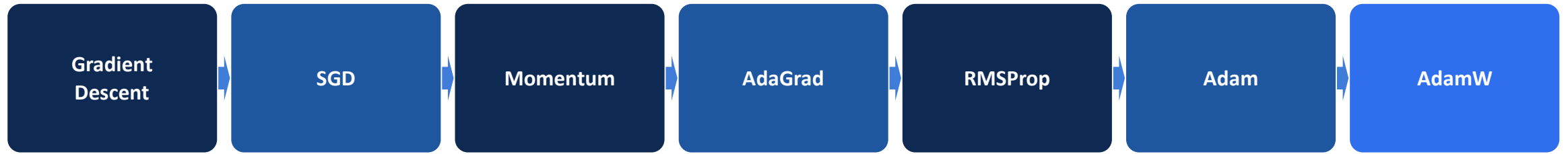


THE KEY CONCEPTUAL DIFFERENCE

How weight decay is applied relative to Adam's adaptive gradient update — decoupled rather than folded into the gradient term.

Widely used in modern deep learning — especially for Transformer models.

The Evolution of Optimizers



NOT A STRICT REPLACEMENT TIMELINE

Each one doesn't completely replace the last. These are related optimization approaches, and several remain useful today.

Each method tries to improve some aspect of how gradients are used to update parameters.

Basic Gradient Descent vs. Adam

BASIC GRADIENT DESCENT

Current weight	0.5
Gradient	-2
Learning rate	0.1

$$w_{\text{new}} = 0.5 - (0.1)(-2) = 0.7$$

Update = +0.2

New weight = 0.7

IF ADAM WERE USED INSTEAD

Adam does not simply compute:

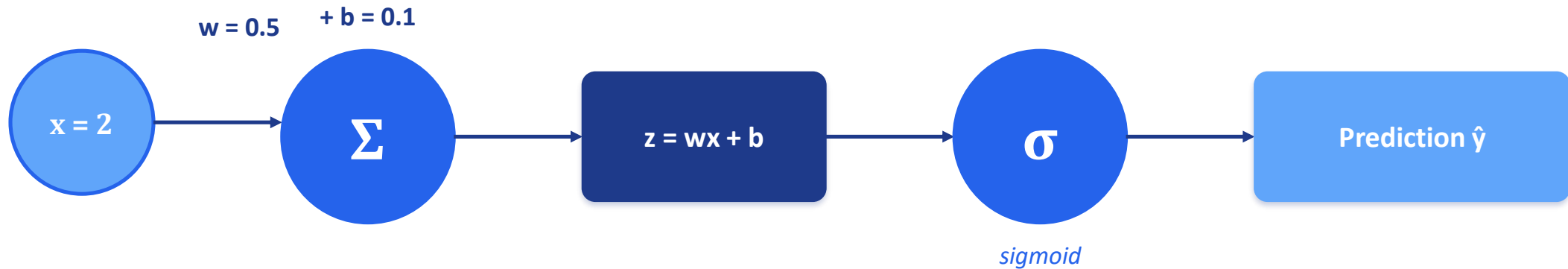
$$weight - learning_rate \times raw_gradient$$

Instead, it uses the history of gradients — their running average and squared average — to shape the update.

That's why Adam is more sophisticated than basic gradient descent.

One Complete Numerical Example

Setup: a single neuron solving binary classification



Given: $x = 2$ $w = 0.5$ $b = 0.1$ true label $y = 1$

Goal: predict the class, measure the error, then update w to reduce that error.

1

Forward Propagation

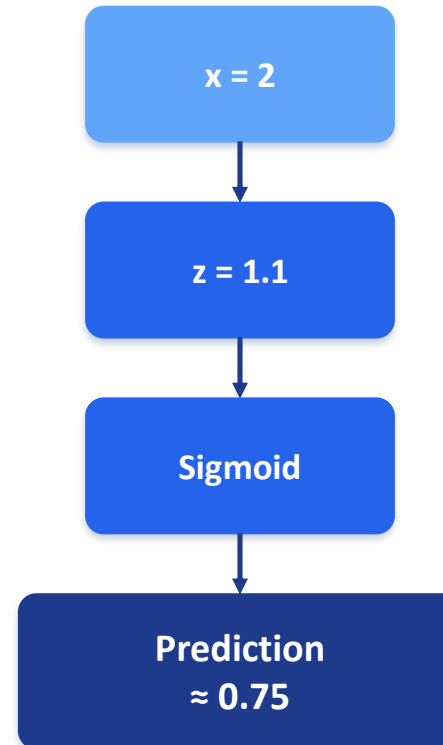
Compute the weighted sum, then apply the activation function

$$z = wx + b$$

$$z = (0.5)(2) + 0.1 = 1.1$$

$$\hat{y} = \sigma(z) = 1 / (1 + e^{-z})$$

$$\hat{y} = \sigma(1.1) \approx 0.75$$



In plain words: the neuron combines its input with the weight and bias, then squashes the result into a probability between 0 and 1 using sigmoid.

2

Compare With the True Label

The loss function measures how wrong the prediction was

TRUE LABEL

$$y = 1$$

PREDICTION

$$\hat{y} = 0.75$$



(75% probability of class 1)

Binary Cross-Entropy Loss:

$$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Since $y = 1$: $L = -\log(0.75)$

$$L \approx 0.288$$

LOSS

$$\approx 0.288$$

3

Backpropagation

How does changing w affect the loss?

We calculate the gradient of the loss with respect to w :

$$\partial L / \partial w = (\hat{y} - y) \cdot x$$

$$= (0.75 - 1) \times 2$$

$$= -0.5$$

A negative gradient means: increasing w would decrease the loss.

GRADIENT

-0.5

*tells the optimizer
which direction to move w*

In plain words: backpropagation traces the error backward through the network to find exactly how much each parameter contributed to it.

4

Optimizer / Gradient Descent

Use the gradient to update the weight

Update rule (learning rate $\eta = 0.1$):

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot (\partial L / \partial w)$$

$$w_{\text{new}} = 0.5 - (0.1)(-0.5)$$

$$w_{\text{new}} = 0.55$$

BEFORE

$$w = 0.50$$



AFTER

$$w = 0.55$$

Why subtract?

The gradient points toward increasing loss.
Moving the weight in the opposite direction
(subtracting) reduces the loss.

The learning rate controls the step size — too large overshoots, too small learns slowly.

5

Forward Again

Same neuron, updated weight — how does the prediction change?

New weight: $w = 0.55$ ($x = 2$, $b = 0.1$ unchanged)

$$z = (0.55)(2) + 0.1 = 1.2$$

$$\hat{y} = \sigma(1.2) \approx 0.769$$

The model became more confident in class 1 — exactly what we want, since the true label was 1.

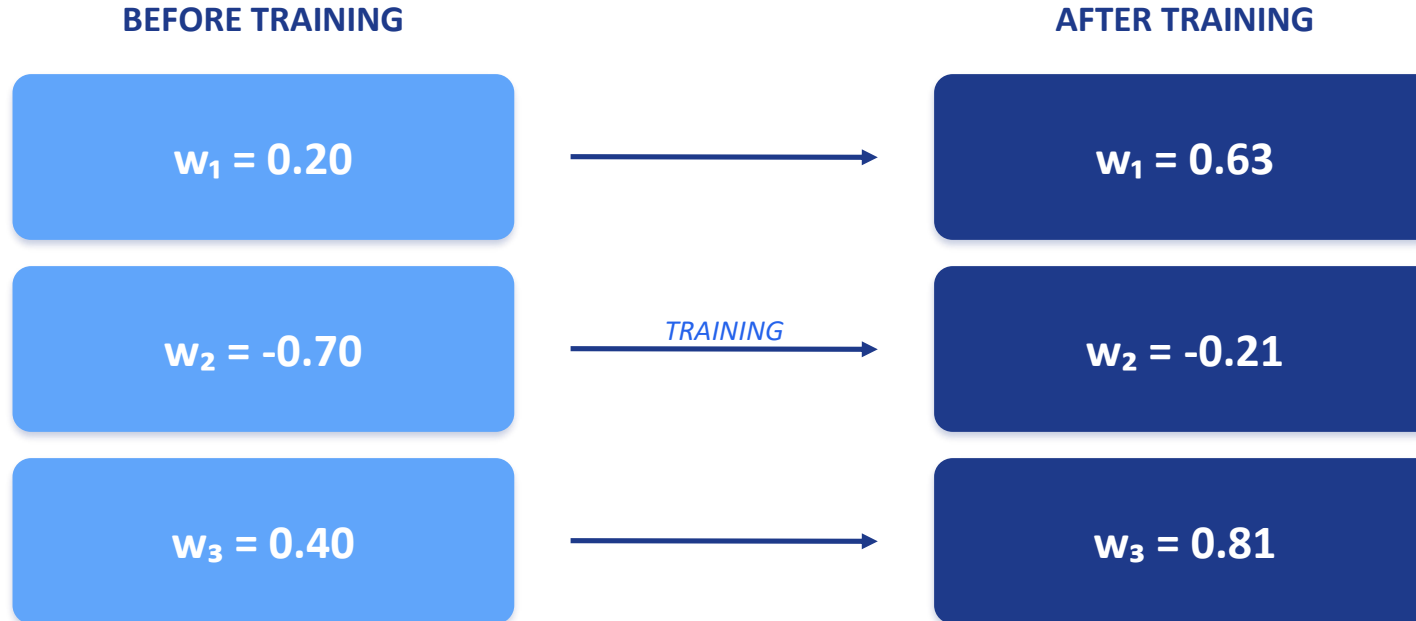
PREDICTION CONFIDENCE



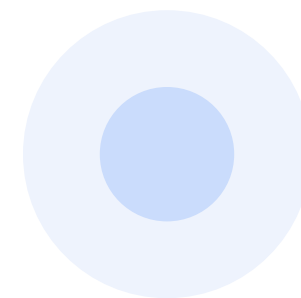
What Actually “Learns”?

Misconception: “the network learns the prediction” — not quite

Learning = finding parameter values (weights & biases) that make the model produce better predictions.



The network's behavior changes only because its parameters change.



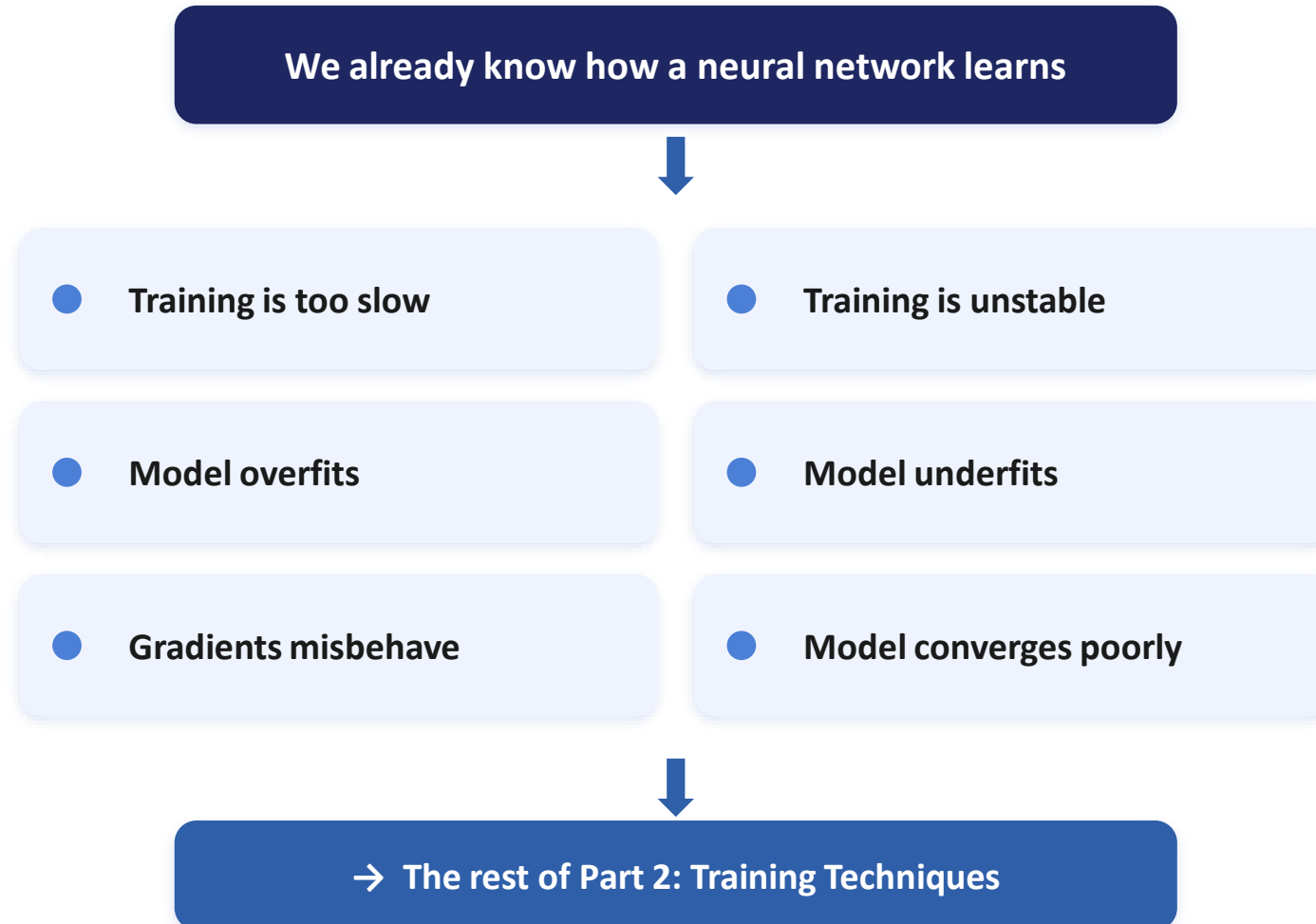
PART 2

Making Neural Networks Train Better

Faster convergence, stable optimization, and less overfitting — the techniques that turn a working network into a well-trained one.

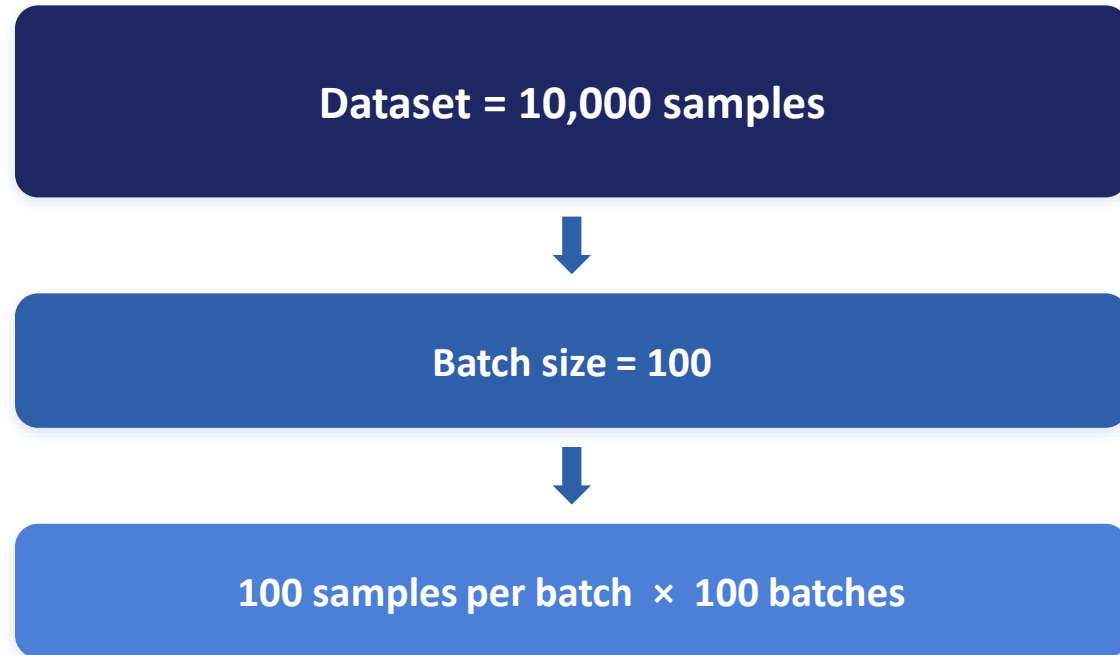


We Know How to Train a Network — But Training Can Have Problems

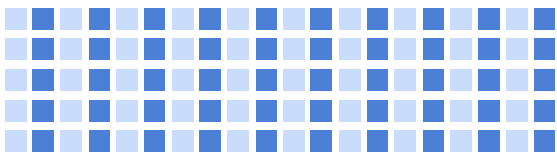


Do We Send All the Data Through at Once?

Not necessarily — we divide the dataset into batches.



100 batches, visualized:



Each batch runs the full training step



Batch, Iteration, and Epoch

1

Batch

A group of training samples

100 samples processed together in one forward/backward pass.

2

Iteration / Step

One parameter update

Batch → Forward → Loss →
Backward → Update.

3

Epoch

One full pass through the data

Batch 1 + Batch 2 + ... + Batch 100 = 1 epoch.

Small Batch vs. Large Batch

Small Batch

Batch size = 16

Advantages

- ✓ Uses less memory
- ✓ More frequent updates
- ✓ Noisier gradients — can help exploration

Disadvantages

- ☐ Less efficient on some hardware
- ☐ Training signal is noisier

Large Batch

Batch size = 512

Advantages

- ✓ More stable gradient estimates
- ✓ Utilizes GPUs efficiently

Disadvantages

- ☐ Needs more memory
- ☐ Fewer updates per epoch
- ☐ Can generalize worse

Batch size is a hyperparameter — it affects memory usage, gradient noise, training speed, and sometimes generalization. Neither choice is universally “better.”

How Many Updates Actually Happen?

Worked Example

Dataset	1,000 samples
Batch size	100
Iterations / epoch	10
Epochs	20

Total parameter updates

200

20 epochs × 10 iterations = 200 updates

?

Does one epoch mean
one weight update?

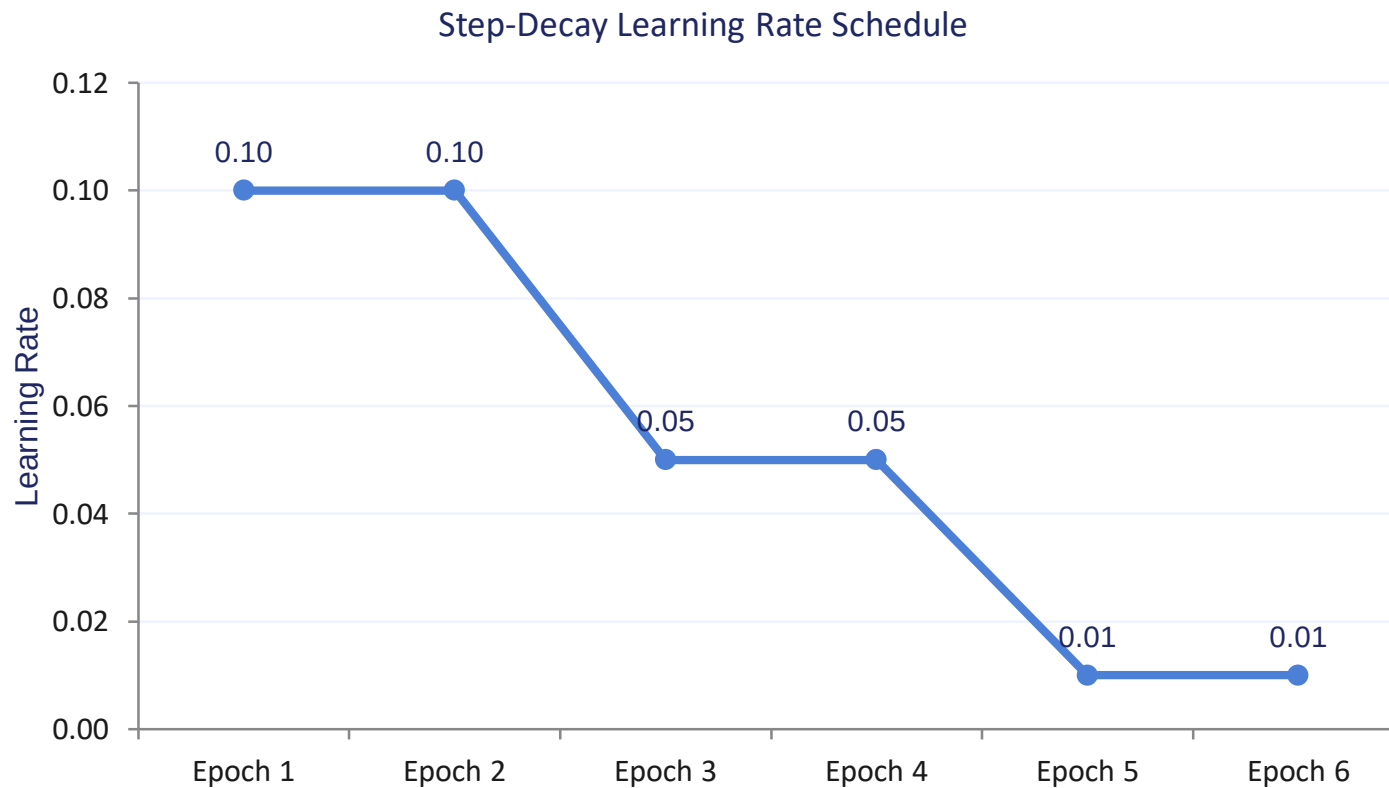
NO.

1 epoch = multiple batches = multiple updates

This is one of the most common beginner misconceptions — an epoch groups many gradient updates, it isn't a single one.

Should the Learning Rate Stay Constant?

Not necessarily — large steps early move quickly toward a good region; smaller steps later allow careful optimization.



Learning-rate scheduling changes the learning rate during training according to a predefined or adaptive strategy.

This directly connects to the learning-rate concept: LR controls step size — scheduling controls how that step size evolves.

Common Schedulers

1

Step Decay

Reduce LR after a fixed number of epochs.

0.1 → 0.01 → 0.001

2

Exponential Decay

Gradually decrease LR every step, following an exponential curve.

LR × decay-rate each step

3

Cosine Annealing

Learning rate follows a smooth, cosine-shaped curve down to a minimum.

High → smooth curve → low

4

ReduceLROnPlateau

Reduce LR only when validation performance stops improving.

Val loss stalls → LR ↓

Before Training Starts: What Are the Weights?

If all weights = 0 ...

- ☐ Neuron 1: $w = 0$
- ☐ Neuron 2: $w = 0$
- ☐ Neuron 3: $w = 0$

Every neuron receives identical inputs and identical parameters — they learn the same thing.

The Symmetry Problem

Random Initialization Fixes It

- ☒ $w_1 = 0.13$
- ☒ $w_2 = -0.42$
- ☒ $w_3 = 0.27$

Now neurons can learn different features. But scale still matters:

Too small
→ Vanishing gradients

Too large
→ Exploding gradients

A Simple Initialization Rule of Thumb

Xavier / Glorot Initialization

Used with: Sigmoid • Tanh

Maintains reasonable variance of activations as information passes through layers.

He Initialization

Used with: ReLU

Accounts for how ReLU zeroes out roughly half of all activations.

Quick Rule

ReLU → He Initialization

Tanh / Sigmoid → Xavier / Glorot

A useful default — modern architectures and frameworks can add further considerations.

Keeping Activations Well-Scaled

Before Normalization

Some activations very large, some very small → training becomes difficult.



After Batch Normalization

Activations are normalized using mini-batch statistics — stable, comparable scale across the network.

$$\hat{\mathbf{x}} = (\mathbf{x} - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$$

$$\mathbf{y} = \gamma \cdot \hat{\mathbf{x}} + \beta$$

μ_B = batch mean · σ^2_B = batch variance · γ, β = learnable scale & shift

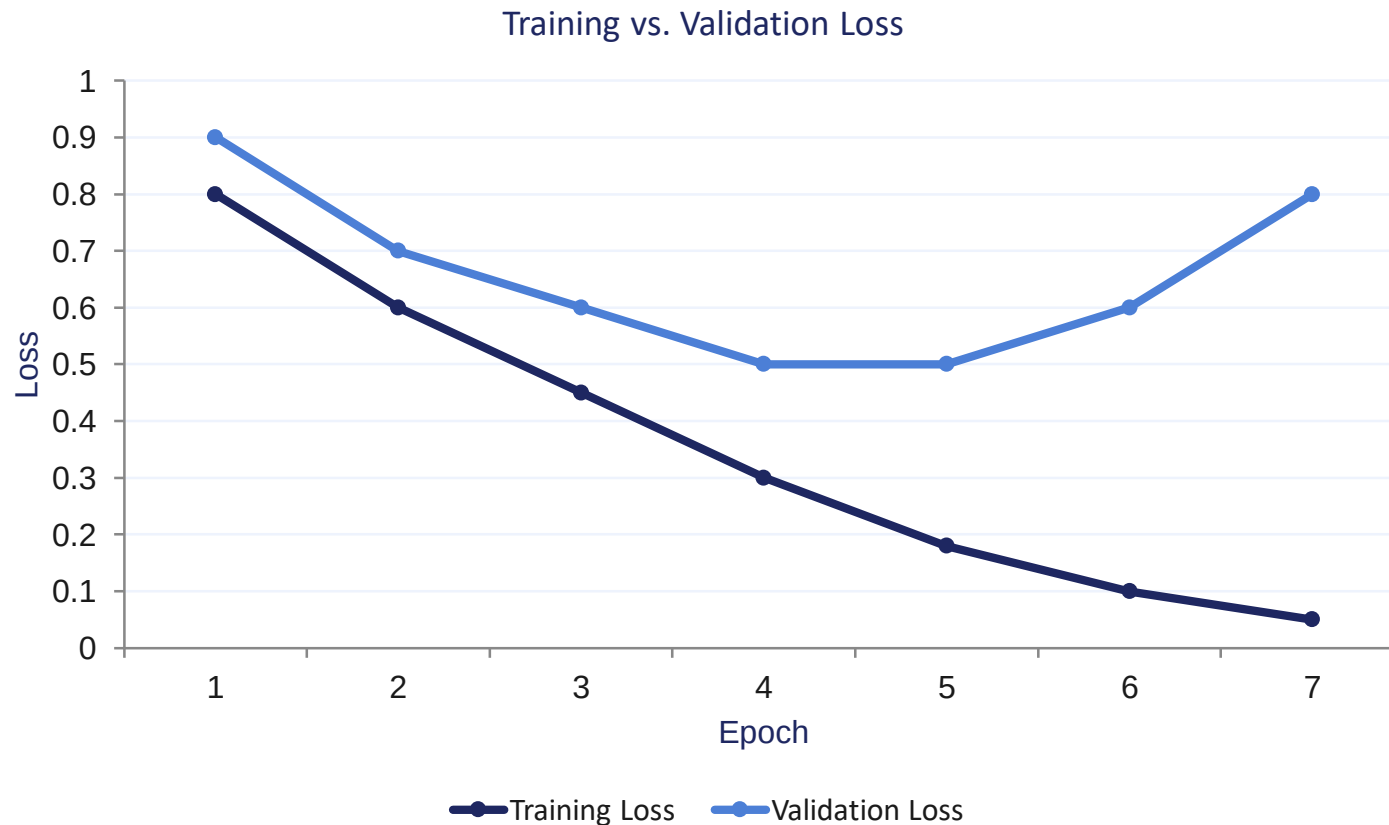
Why use it?

- ✓ Stabilizes training
- ✓ Allows larger learning rates in some cases
- ✓ Can provide a mild regularization effect

Caution

BatchNorm helps a great deal, but it does not completely solve vanishing or exploding gradients.

When the Model Memorizes the Training Data



Overfitting

The model performs extremely well on training data but poorly on unseen data — it has learned the training set too specifically.

Training accuracy

99%

Validation accuracy

75%

Pattern to watch for: training loss keeps dropping while validation loss starts climbing back up.

When the Model Is Too Simple to Learn

Underfitting

Training acc = 70% · Validation acc = 68% — the model isn't even performing well on data it has already seen.

	Underfitting	Good Fit	Overfitting
Training performance	Poor	Good	Very good
Validation performance	Poor	Good	Poor
Training loss	High	Low	Very low
Validation loss	High	Low	Starts increasing

How Can We Reduce Overfitting?

Regularization discourages the model from relying on overly complex parameter configurations.

L2 Regularization

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum w_i^2$$

Penalizes squared weights. Encourages smaller weights overall. Closely related to weight decay.

Rarely produces exact zeros

L1 Regularization

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum |w_i|$$

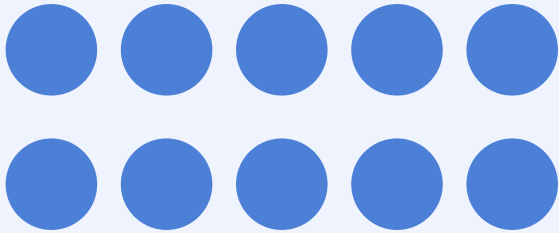
Penalizes absolute weight values. Pushes some weights to exactly zero, producing a sparse model.

Useful for feature selection

Large weights → large penalty → the optimizer is nudged toward smaller, simpler weight configurations — reducing the model's tendency to overfit.

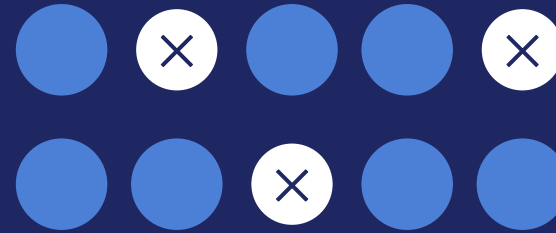
Randomly Disabling Neurons During Training

Before Dropout — all neurons active



Every neuron participates every step.

One training step — Dropout ON



A random subset is disabled each step — the network can't lean on any single neuron.

Training

Dropout: ON

Forces the network to build robust, distributed representations instead of depending on specific neurons.

Inference

Dropout: OFF

All neurons are used for predictions. Frameworks like PyTorch switch this automatically between train/eval modes.

Stop Training Before Overfitting Gets Worse

Epoch	Train Loss	Val Loss
1	0.8	0.9
2	0.6	0.7
3	0.5	0.6
4	0.4	0.5
5	0.3	0.5
6	0.2	0.6
7	0.1	0.8

Best validation loss occurs at epoch 4 (0.5) — rows after that show no further improvement.

Early stopping stops training when validation performance stops improving, according to a chosen criterion.

We usually don't stop after just one bad epoch — we use patience.

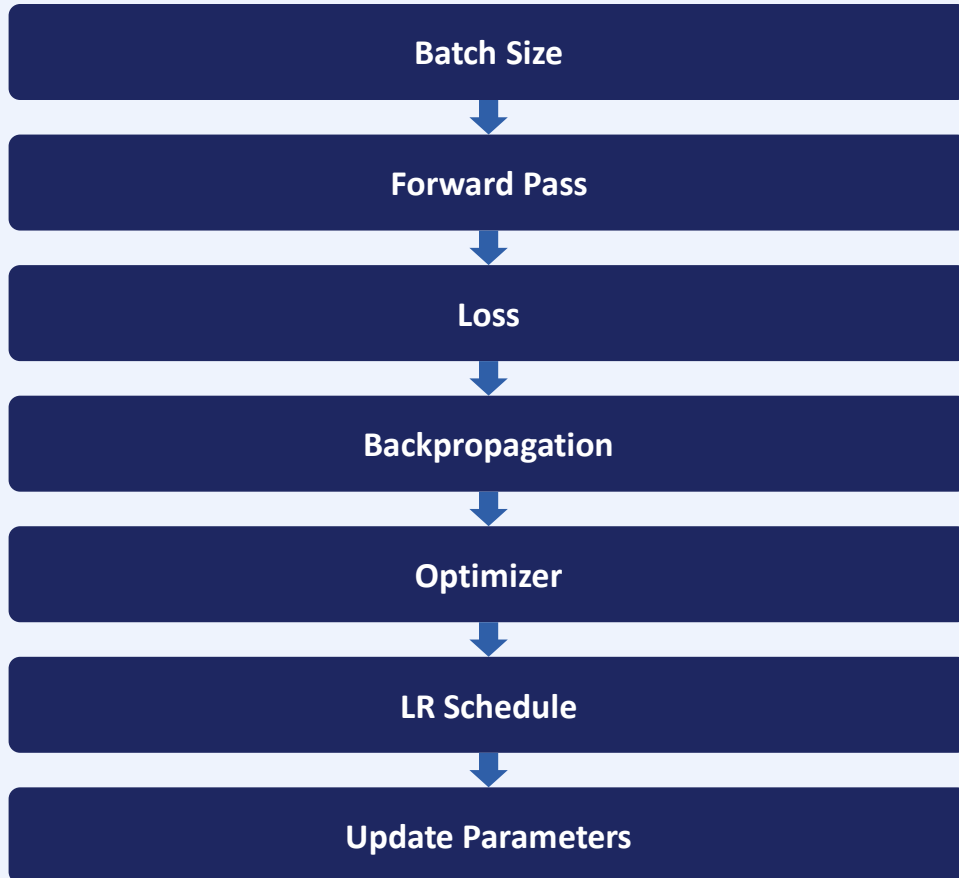
Patience = 3

- Epoch 10 — Best val loss
- Epoch 11 — No improvement
- Epoch 12 — No improvement
- **Epoch 13 — No improvement → STOP**

Tip: restore the best checkpoint, not the parameters from the final epoch.

How Everything Fits Together

The Training Loop



Techniques That Help

- **Weight Initialization** → *good starting point*
- **Batch Normalization** → *more stable optimization*
- **LR Scheduling** → *better optimization over time*
- **L1 / L2** → *regularization*
- **Dropout** → *regularization*
- **Early Stopping** → *stop before excess overfitting*

Optimization vs. Generalization

Help Us Optimize / Train

Batch Size · Learning Rate · LR Scheduling · Weight Initialization ·
Batch Normalization · Optimizers

Help Us Generalize

L1 · L2 · Dropout · Early Stopping

Overfitting and Underfitting are the problems we diagnose along the way.

Quick Troubleshooting Guide

Training too slow

- Learning rate
- Batch size
- Optimizer choice
- LR scheduler

Training unstable

- Learning rate
- Weight initialization
- Batch Normalization
- Optimizer choice

Model overfits

- Dropout
- L1 / L2 regularization
- Early stopping
- More data / augmentation

Model underfits

- More model capacity
- Better features / architecture
- Train longer
- Reduce regularization, tune LR